

DeepSeek R1: Architecture and Training, End to End

Contents

DeepSeek R1: Architecture and Training, End to End	1
Part 1 — How DeepSeek-R1 Is Trained	1
Part 2 — The Five Architectural Innovations	3
Part 3 — Full Worked Example: “<bos> I love AI” Through One Decoder Layer	6
Appendix A — Fully Expanded Per-Token Arithmetic, in LaTeX Matrix Form: “<bos> I love AI”	8

DeepSeek R1: Architecture and Training, End to End

A complete reference: how DeepSeek-R1 is trained (GRPO and the multi-stage pipeline), the five architectural innovations that make DeepSeek-V3/R1 different from a standard Transformer (Multi-Head Latent Attention, Rotary Positional Embedding, Mixture of Experts, Multi-Token Prediction, and FP8 quantization), and a complete, fully-expanded worked numerical example of one decoder layer processing a real sentence, from tokenization through two predicted future tokens.

Part 1 — How DeepSeek-R1 Is Trained

The base model

R1 doesn’t start from scratch — it fine-tunes **DeepSeek-V3-Base**, a 671B-parameter Mixture-of-Experts transformer (37B activated per token), 61 layers, hidden dimension 7168, built on Multi-Head Latent Attention and DeepSeekMoE (both covered in full in Part 2).

The central question: can reasoning emerge from pure RL, with no human-written reasoning examples?

Every prior “reasoning” LLM was trained by imitating human-written chain-of-thought. DeepSeek’s experiment: what if a model is only ever rewarded for getting the final answer right, and left to invent its own reasoning style through trial and error, with zero supervised fine-tuning first? That model is **DeepSeek-R1-Zero**.

The algorithm: GRPO (Group Relative Policy Optimization)

Standard RLHF (PPO) needs a separate **value network** to estimate how good a partial response is likely to be — a second, similarly-sized model, doubling memory and adding real instability for reasoning traces that can run thousands of tokens with reflection and revision.

GRPO’s trick: throw away the value network. For a given question, sample a **group** of G outputs from the current policy, score each with the reward function, and use the **group’s own mean and standard deviation** as the baseline:

$$A_i = \frac{r_i - \text{mean}(\{r_1, \dots, r_G\})}{\text{std}(\{r_1, \dots, r_G\})}$$

No learned critic — if a response scored above the group average it gets a positive advantage (encouraged), below average a negative one (discouraged). Full objective (closely related to PPO’s clipped objective):

$$J_{GRPO}(\theta) = \mathbb{E} \left[\frac{1}{G} \sum_{i=1}^G \min(\rho_i A_i, \text{clip}(\rho_i, 1 - \epsilon, 1 + \epsilon) A_i) - \beta D_{KL}(\pi_\theta \| \pi_{ref}) \right]$$

$$\text{where } \rho_i = \frac{\pi_\theta(o_i|q)}{\pi_{\theta_{old}}(o_i|q)}$$

- The ratio ρ_i is the same importance-sampling correction PPO uses, letting sampled rollouts be reused for more than one gradient step.
- $\text{clip}(\cdot, 1 - \epsilon, 1 + \epsilon)$ prevents any single update from moving the policy too far in one step (DeepSeek used $\epsilon = 10$ for R1 — a notably large clip range).
- $\beta D_{KL}(\pi_\theta \| \pi_{ref})$ penalizes drifting too far from a fixed reference policy, added directly to the loss (unlike PPO, which folds KL into the per-token reward).

Worked example of the advantage calculation: for one math question, the model samples $G = 8$ responses at temperature 1.0. Reward is binary (1 if the final answer is correct, 0 if not) — say 3 of 8 got it right:

$$\begin{aligned} \text{rewards} &= [1, 1, 1, 0, 0, 0, 0, 0] & \text{mean} &= \frac{3}{8} = 0.375 \\ \text{std} &= \sqrt{\frac{(1 - 0.375)^2 \times 3 + (0 - 0.375)^2 \times 5}{8}} = \sqrt{\frac{1.1719 + 0.7031}{8}} = \sqrt{0.2344} = 0.4841 \end{aligned}$$

For a correct response: $A = \frac{1 - 0.375}{0.4841} = 1.291$ (positive — increase its probability)

For an incorrect response: $A = \frac{0 - 0.375}{0.4841} = -0.775$ (negative — decrease its probability)

If all 8 responses in a group got the *same* reward, $\text{std} = 0$ and the advantage is undefined/zero — that group contributes no learning signal at all. This is a real, documented weakness of GRPO, though it’s rare with sufficiently hard problems.

Reward design — deliberately simple, rule-based, no learned reward model

$$\text{Reward} = \text{Reward}_{\text{accuracy}} + \text{Reward}_{\text{format}}$$

- **Accuracy reward** — for math, does the final boxed answer match ground truth exactly (checked by a symbolic parser)? For code, do the outputs pass the test suite when compiled and run? Binary, deterministic. - **Format reward** — did the model wrap its reasoning in `<think>...</think>` tags before `<answer>...</answer>`? The only structural constraint — reasoning *content* is left completely unconstrained.

DeepSeek deliberately avoided a neural reward model for reasoning tasks: a learned reward model can itself be gamed (“reward hacking”) once large-scale RL pushes against it long enough. Rule-based rewards (an exact-match checker, a compiler) can’t be exploited the same way.

What happened: emergent behavior, no imitation

With nothing but this signal, R1-Zero’s accuracy on AIME 2024 rose from 15.6% to 77.9% over training, and the model spontaneously began generating longer responses, re-deriving intermediate steps, checking its own work, and backtracking mid-solution (“Wait, that’s not right, let me reconsider”) — none of which were ever demonstrated to it. DeepSeek calls this emergence the **“aha moment.”** Nobody wrote a reward for self-correction; it emerged because responses containing it happened to get the final answer right more often, and group-relative advantage rewarded that pattern into existence.

The catch: R1-Zero is genuinely unpleasant to read — it mixes English and Chinese mid-reasoning and has poor formatting, because nothing in the reward function ever asked for readability.

DeepSeek-R1: a 4-stage pipeline to fix readability

Stage 1 — Cold start: a few thousand human-curated long chain-of-thought examples, used for ordinary SFT, to give the model a readable, consistent “first-person, one-language” reasoning style before RL even starts.

Stage 2 — Reasoning-oriented RL: same GRPO + accuracy/format rewards as R1-Zero, plus a new language-consistency reward (rewards responses that stay in one target language):

$$\text{Reward} = \text{Reward}_{rule} + \text{Reward}_{language}$$

Stage 3 — Rejection sampling + SFT (~800k examples): sample many responses from the Stage-2 checkpoint, keep only the ones that got the right answer AND read cleanly, mix in ~200k general (non-reasoning) examples for writing/QA/translation, and do ordinary supervised fine-tuning on the combined 800k set.

Stage 4 — Full RL (reasoning + human preference): GRPO again, combining the rule-based reasoning reward with a trained preference reward model for helpfulness/harmlessness:

$$\text{Reward} = \text{Reward}_{reasoning} + \text{Reward}_{general} + \text{Reward}_{language}$$

$$\text{where } \text{Reward}_{general} = \text{Reward}_{RM} + \text{Reward}_{format}$$

Each stage patches a specific failure of the previous one: Stage 1 fixes unreadable style before RL even starts; Stage 2 fixes language-mixing; Stage 3 broadens the model past pure reasoning into general usefulness; Stage 4 aligns it to human preference without losing the reasoning gains.

Distillation into smaller models

Rather than running this whole RL pipeline on smaller models from scratch, DeepSeek used **R1’s own generated reasoning traces** (the 800k SFT set from Stage 3) as ordinary supervised fine-tuning data for existing smaller open models (Qwen and Llama checkpoints, 1.5B to 70B) — plain SFT, no RL. These distilled models ended up beating other similarly-sized models that *were* trained with RL directly — a large model’s self-discovered reasoning patterns transfer better through imitation than a small model can discover on its own through RL.

Part 2 — The Five Architectural Innovations

The parts of a standard Transformer that stay the same: token embeddings, stacked attention + feed-forward blocks with residual connections and normalization, and a final linear+softmax over the vocabulary to predict the next token. What’s different in DeepSeek-V3/R1 is *how* attention is computed (MLA), *how* position is encoded (RoPE), *how* the feed-forward block is structured (MoE), *what* the training objective predicts (MTP), and *what number format* the arithmetic runs in (FP8).

1. Multi-Head Latent Attention (MLA)

The problem: standard multi-head attention caches a full-size Key and Value vector per head, per token, for the entire context — at long sequence lengths this KV cache dominates inference memory. MLA compresses K and V into a small shared **latent vector** before caching, then reconstructs full-size K/V from it on demand.

The math:

$$c_{KV} = h_t W^{DKV} \quad (\text{down-project: } d_{model} \rightarrow d_c, \text{ a small latent, e.g. 512-dim})$$

$$K^C = c_{KV} W^{UK} \quad (\text{up-project latent back to full per-head K})$$

$$V^C = c_{KV} W^{UV} \quad (\text{up-project latent back to full per-head V})$$

Only c_{KV} needs to be cached per token, not the full $n_h \times d_h$ K and V matrices.

Worked toy example: $d_{model} = 4$, latent dim $d_c = 2$, 1 head with $d_h = 2$.

$$h_t = [0.5, -0.3, 0.2, 0.1] \quad W^{DKV} = \begin{bmatrix} 0.3 & 0.1 \\ 0.2 & -0.2 \\ 0.1 & 0.3 \\ 0.0 & 0.2 \end{bmatrix}$$

$$c_{KV} = h_t W^{DKV} = [0.15 - 0.06 + 0.02 + 0, 0.05 + 0.06 + 0.06 + 0.02] = [0.11, 0.19]$$

Just 2 numbers get cached, instead of a separate K and V. Real DeepSeek-V3 (128 heads $\times$ 128 dims $\rightarrow$ a ~ 512 -dim latent) achieves roughly a 93% reduction in KV cache size versus standard multi-head attention. A small “decoupled” positional key component (carrying RoPE) is computed separately and concatenated onto K^C , because RoPE’s rotation doesn’t commute cleanly with the compression.

2. Rotary Positional Embedding (RoPE)

The problem: adding a learned position vector to the token embedding (BERT’s approach) doesn’t generalize well to longer sequences and doesn’t cleanly encode *relative* distance. RoPE instead **rotates** the Query and Key vectors by an angle proportional to position.

The math: split dimensions into 2D pairs. For pair index i (out of $d/2$ pairs) at position m :

$$\theta_i = 10000^{-2i/d} \quad \text{angle} = m\theta_i$$

$$\begin{bmatrix} x'_1 \\ x'_2 \end{bmatrix} = \begin{bmatrix} \cos(\text{angle}) & -\sin(\text{angle}) \\ \sin(\text{angle}) & \cos(\text{angle}) \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \end{bmatrix}$$

Early dimension-pairs rotate fast (fine-grained local position), later pairs rotate slowly (coarse, long-range position). The dot product of a rotated query at position m and rotated key at position n depends only on $(m - n)$ — the relative distance, not absolute position — which is what lets rotary-based models extrapolate more gracefully to longer contexts.

Worked example ($d = 4$, pair index $i = 0$, $\theta_0 = 1$):

$$x = [1.0, 0.5]$$

$$\begin{aligned} \text{pos} = 0 : \text{angle} = 0 &\Rightarrow \text{rotated} = [1.0000, 0.5000] \\ \text{pos} = 1 : \text{angle} = 1 &\Rightarrow \text{rotated} = [0.1196, 1.1116] \\ \text{pos} = 2 : \text{angle} = 2 &\Rightarrow \text{rotated} = [-0.8708, 0.7012] \end{aligned}$$

Vector length is preserved at every position; only direction changes, and that direction change encodes distance into the sequence.

Interaction with MLA: since RoPE needs to rotate the actual K vector but MLA wants to cache only the compressed latent, DeepSeek-V3 splits each head’s key into a compressed content part (K^C , no RoPE) and a small separate “decoupled” part that does carry RoPE (K^R , shared across all heads), concatenated before the attention dot product.

3. Mixture of Experts (MoE) — DeepSeekMoE with auxiliary-loss-free balancing

The problem: a dense feed-forward layer pushes every token through the same full-size FFN regardless of how specialized the input is. MoE has many smaller expert FFNs and routes each token to only a handful.

The math:

$$s_i = h_t \cdot e_i \quad (\text{routing score: affinity between token } h_t \text{ and expert centroid } e_i)$$

$$\text{biased score (routing only)} = s_i + b_i \quad (b_i \text{ NOT used in final weighting})$$

$$g_i = \frac{\text{sigmoid}(s_i)}{\sum_{j \in \text{selected}} \text{sigmoid}(s_j)} \quad (\text{gate value, normalized over selected experts, using ORIGINAL unbiased score})$$

$$h'_t = h_t + \text{FFN}_{\text{shared}}(h_t) + \sum_{i \in \text{selected}} g_i \cdot \text{FFN}_i(h_t)$$

The bias b_i is the “auxiliary-loss-free” trick: instead of an extra loss term fighting the main training objective to keep experts balanced, DeepSeek nudges b_i up after a batch if expert i was under-used, and down if over-used — purely a routing-time adjustment, invisible to the gradient computation for the main loss.

Worked example: 4 routed experts, top-2 selected, toy 4-dim hidden state:

$$h_t = [0.5, -0.2, 0.3, 0.1] \quad \text{raw scores } s_i : [0.45, -0.06, 0.32, 0.18]$$

$$\text{bias } b = [0.0, 0.05, -0.02, 0.10] \quad \text{biased scores: } [0.45, -0.01, 0.30, 0.28]$$

Top-2 by biased score: Expert 0 (0.45) and Expert 2 (0.30). Expert 3 (biased score 0.28) just misses the cut versus expert 2 (0.30) — this is exactly where the bias term earns its keep: if expert 3 has been historically under-used, its bias climbs over successive batches until it starts getting picked more often, without touching the training loss.

$$\begin{aligned} \text{sigmoid}(0.45) &= 0.6106, & \text{sigmoid}(0.32) &= 0.5793 \\ g_0 &= \frac{0.6106}{0.6106 + 0.5793} = 0.5132, & g_2 &= \frac{0.5793}{0.6106 + 0.5793} = 0.4868 \end{aligned}$$

Only 2 of the 4 expert FFNs actually ran. At DeepSeek-V3’s real scale (256 routed experts, 8 selected, plus 1 shared) this is what lets a 671B-parameter model activate only 37B parameters per token.

4. Multi-Token Prediction (MTP)

The problem: standard training only ever asks “predict the very next token.” MTP adds extra training signal by also predicting tokens 2, 3, ... steps ahead, using cheap sequential extra modules, densifying the gradient signal and pushing the model toward representations that “plan ahead.”

The math, for the MTP module at depth k (predicting token $i + k$):

$$\begin{aligned} h_i'^{(k)} &= M_k \cdot [\text{RMSNorm}(h_i^{(k-1)}); \text{RMSNorm}(\text{Emb}(t_{i+k}))] \\ h_i^{(k)} &= \text{TRM}_k(h_i'^{(k)}) \quad (\text{a lightweight transformer block, depth-specific}) \\ P_i^{(k)} &= \text{Out}(h_i^{(k)}) \quad (\text{shared output head, same as main model}) \end{aligned}$$

This is sequential, not parallel: depth-2’s module consumes depth-1’s output, preserving a genuine causal chain between predicted tokens.

Worked example: input tokens $[t_1, t_2, t_3, t_4]$, main model predicts t_5 (depth 0). MTP module 1 ($k = 1$) takes the main model’s hidden state, concatenates it with the ground-truth embedding of t_5 , and predicts t_6 :

$$h_4'^{(1)} = M_1 \cdot [\text{RMSNorm}(h_4^{(0)}); \text{RMSNorm}(\text{Emb}(t_5))]$$

Total training loss: $\mathcal{L} = \mathcal{L}_{\text{main}} + \lambda_1 \mathcal{L}_{\text{MTP}}^{(1)} + \dots$ (DeepSeek-V3 uses depth 1, predicting 2 tokens ahead total). At inference these modules can be discarded (zero cost to ordinary generation) or repurposed for **speculative decoding**: the lightweight MTP module drafts extra tokens, the full model verifies them in parallel — DeepSeek reports roughly 1.8x throughput from this.

5. Quantization — FP8 mixed-precision training

The problem: most training compute goes into matrix multiplications (GEMMs). Running these in 8-bit floating point (FP8) instead of 16/32-bit is much faster and uses less memory — but FP8 has a tiny dynamic range (roughly ± 448 for the E4M3 format DeepSeek uses), so one large outlier can force everything else in a naively-scaled tensor toward zero.

The math — fine-grained block scaling: instead of one scale factor per whole tensor, DeepSeek computes a separate scale factor per small block: a 1×128 “tile” for activations, a 128×128 “block” for weights.

$$\text{scale} = \frac{\text{FP8}_{\max}}{\max(|\text{block}|)} \quad (\text{FP8}_{\max} \approx 448 \text{ for E4M3})$$

$$\text{quantized value} = \text{round_to_fp8}(\text{original value} \times \text{scale}) \quad \text{dequantized} = \frac{\text{quantized value}}{\text{scale}}$$

Accumulation (the actual summing in a matmul) is still done in higher precision (FP32) even though multiplication happens in FP8, to avoid compounding rounding error.

Worked example — an 8-element toy block (real blocks are 128 elements):

$$\text{block} = [0.012, -1.35, 0.87, 4.2, -3.9, 0.003, 2.1, -0.5]$$

$$\text{amax} = 4.2, \quad \text{scale} = \frac{448}{4.2} = 106.6667$$

$$\text{dequantized back} = [0.0117, -1.35, 0.90, 4.2, -3.9, 0.0029, 2.1, -0.4875]$$

$$\text{absolute error vs. original} = [0.0003, 0.0, 0.03, 0.0, 0.0, 0.0001, 0.0, 0.0125]$$

The largest error (0.03, on 0.87) happens on a mid-sized value, not the largest one — the block’s own scale factor is tuned to its own max, so that value quantizes almost perfectly while values further from the max lose more relative precision. This is exactly why block-wise (rather than whole-tensor) scaling matters: it keeps one block’s outlier from wrecking precision for unrelated blocks of the same tensor.

What else belongs on this list

- **GRPO** (Part 1) — arguably the single most R1-specific innovation; the other five are properties of the base model V3.
- **DualPipe pipeline parallelism** — a training-systems scheduling technique overlapping forward/backward computation and communication across GPUs, designed to hide communication latency in the MoE’s cross-node expert routing. Pure systems engineering, not a math concept, but a named contribution alongside FP8 and MoE.
- **The auxiliary-loss-free bias mechanism** — folded into the MoE section above since it’s the load-balancing half of that story, but worth knowing as its own named idea.

Part 3 — Full Worked Example: “<bos> I love AI” Through One Decoder Layer

Sequence: “<bos> I love AI” (4 tokens). Toy dims: $d_{\text{model}} = 4$, 2 heads. Unlike BERT, this is **decoder-only and causal** — token i can only attend to tokens $0..i$, never anything after it, because the whole point is autoregressive generation. This section walks the narrative once at a readable pace; **Appendix A** repeats every one of these steps with the arithmetic fully expanded in LaTeX matrix form for all 4 tokens.

Tokenization and embedding — the mechanism itself

A real tokenizer (DeepSeek uses a BPE — Byte-Pair Encoding — tokenizer, ~128K vocabulary) doesn’t work word-by-word. It starts from raw bytes/characters and repeatedly merges the most frequent adjacent pair into a new token, built from statistics over a huge training corpus, until it has a fixed vocabulary of ~100K+ merged units. At inference time, a sentence gets tokenized in one pass: greedily match the longest

known vocabulary piece at each position. So “I love AI” might become tokens like ["I", " love", " AI"] — each an entry in a giant lookup table with its own integer ID.

The **embedding matrix** is a single learned table of shape $[\text{vocab_size}, d_{\text{model}}]$ — e.g. $[128000, 7168]$ in real DeepSeek-V3. Looking up a token’s embedding is nothing more than indexing one row of this matrix by its integer ID. No computation is involved — it’s a memory lookup, and it’s the *same* matrix (transposed) that gets reused at the very end as the “unembedding”/output head to turn a final hidden state back into vocabulary logits.

No positional or segment vector is summed in here, unlike BERT — position is injected via **rotation** inside attention (RoPE), not addition to the input.

Step 3: RMSNorm (pre-norm, before attention — not after, like BERT)

$$\text{RMSNorm}(x) = \frac{x}{\sqrt{\text{mean}(x^2) + \epsilon}} \quad (\text{no mean-centering, unlike LayerNorm})$$

This normalized version — not the raw embedding — is what feeds attention. “Pre-norm” is what essentially every modern decoder (Llama, DeepSeek, GPT-style models) uses instead of BERT’s post-norm.

Steps 4-8: MLA — compress into latents, reconstruct per-head content, apply RoPE

Down-project into small shared latents (this is literally the only thing cached during real inference), up-project each head’s own small content (“nope” = “no positional info”) slice from these latents into $K_{\text{content}}, Q_{\text{content}}, V$. Separately project a small “rope-carrying” piece of Q and K (shared across both heads), then **rotate** it by an angle proportional to position. This rotation is literally the only place position enters the model. Assemble the full per-head Q and K as $[\text{content}; \text{rotated rope}]$.

Step 9: Causal scaled dot-product attention

Same score $= QK^T / \sqrt{d_k}$ and softmax as BERT, but with a **causal mask** first — any score where key-position $>$ query-position gets set to $-\infty$ before softmax. The resulting softmax weight matrix has a strict staircase shape: the first token can only attend to itself, the second to itself and the first, and so on — this is what “causal” looks like numerically.

Steps 10-12: Concat heads, project, residual, second RMSNorm

Concatenate every head’s output, project through W_o , add back to the *original* (pre-normalization) residual stream X — pre-norm architectures always add back to the pre-norm residual, not the normalized version — then apply a second RMSNorm before the MoE block.

Step 13: MoE routing

Score every token against all 4 toy experts, add the auxiliary-loss-free bias for the routing *decision* only, pick the top-2, then compute gate values from the sigmoid of the **original unbiased** scores, renormalized over just the two selected experts. Every token in the sentence routes to a different pair of experts — exactly the per-token specialization MoE exists to enable.

Steps 14-16: Shared + routed expert FFNs, combine

The shared expert runs for every token, no exception. Each token’s own selected pair of routed experts also runs, and their outputs are combined with the gate weights computed above:

$$\text{Full MoE output} = Y(\text{residual}) + \text{shared_expert}(Y_n) + \sum \text{gated routed experts}$$

Step 17: Final RMSNorm — end of this decoder layer

In real DeepSeek-V3 this whole sequence (Steps 3-17) repeats 61 times.

Step 18: Main-model next-token prediction

Every position’s final hidden state predicts the token that should come right after it — project through the tied embedding/output matrix, softmax over the vocabulary. The same embedding table is used at both ends of the network (weight tying): a word’s embedding and a position’s final hidden state live in the same vector space, so predicting a word is really asking which word’s own embedding vector the hidden state points most closely toward. **Appendix A** shows the complete five-step softmax breakdown (find max, shift, exponentiate, sum, divide) for every position.

Step 19: MTP module (depth 1) — predict the token *after that*

Combine the depth-0 hidden state with the **true** embedding of the next token (teacher forcing), project through a small depth-specific module, and predict two tokens ahead — sequentially and causally, not as an independent guess.

Step 20: FP8 quantization, applied to a real weight matrix from this walkthrough

Quantize W_o itself (from Step 10) using block scaling, then re-run the actual Step-10 matmul with the quantized weights — showing the small, controlled numerical error that is FP8’s entire trade-off, made concrete on a real number from this exact walkthrough.

Appendix A — Fully Expanded Per-Token Arithmetic, in LaTeX Matrix Form: “<bos> I love AI”

X (token embedding lookup) (shape 4×4):

<bos>	0.0500	0.0500	0.0500	0.0500
I	0.2000	0.1000	−0.1000	0.0000
love	−0.1000	0.3000	0.2000	0.1000
AI	0.3000	−0.2000	0.1000	0.2000

Step 3: RMSNorm on every token

Input matrix (every token as a row) (shape 4×4):

<bos>	0.0500	0.0500	0.0500	0.0500
I	0.2000	0.1000	−0.1000	0.0000
love	−0.1000	0.3000	0.2000	0.1000
AI	0.3000	−0.2000	0.1000	0.2000

<bos>: $v = [0.0500, 0.0500, 0.0500, 0.0500]$

$$\text{mean}(v^2) = \frac{0.0025 + 0.0025 + 0.0025 + 0.0025}{4} = \frac{0.0100}{4} = 0.0025$$

$$\text{denom} = \sqrt{0.0025 + 10^{-6}} = 0.0500$$

$$\text{normalized} = \left[\frac{0.0500}{0.0500} = 0.9998, \frac{0.0500}{0.0500} = 0.9998, \frac{0.0500}{0.0500} = 0.9998, \frac{0.0500}{0.0500} = 0.9998 \right]$$

I: $v = [0.2000, 0.1000, -0.1000, 0.0000]$

$$\text{mean}(v^2) = \frac{0.0400 + 0.0100 + 0.0100 + 0.0000}{4} = \frac{0.0600}{4} = 0.0150$$

$$\text{denom} = \sqrt{0.0150 + 10^{-6}} = 0.1225$$

$$\text{normalized} = [\frac{0.2000}{0.1225} = 1.6329, \frac{0.1000}{0.1225} = 0.8165, \frac{-0.1000}{0.1225} = -0.8165, \frac{0.0000}{0.1225} = 0.0000]$$

$$\text{love: } v = [-0.1000, 0.3000, 0.2000, 0.1000]$$

$$\text{mean}(v^2) = \frac{0.0100 + 0.0900 + 0.0400 + 0.0100}{4} = \frac{0.1500}{4} = 0.0375$$

$$\text{denom} = \sqrt{0.0375 + 10^{-6}} = 0.1937$$

$$\text{normalized} = [\frac{-0.1000}{0.1937} = -0.5164, \frac{0.3000}{0.1937} = 1.5492, \frac{0.2000}{0.1937} = 1.0328, \frac{0.1000}{0.1937} = 0.5164]$$

$$\text{AI: } v = [0.3000, -0.2000, 0.1000, 0.2000]$$

$$\text{mean}(v^2) = \frac{0.0900 + 0.0400 + 0.0100 + 0.0400}{4} = \frac{0.1800}{4} = 0.0450$$

$$\text{denom} = \sqrt{0.0450 + 10^{-6}} = 0.2121$$

$$\text{normalized} = [\frac{0.3000}{0.2121} = 1.4142, \frac{-0.2000}{0.2121} = -0.9428, \frac{0.1000}{0.2121} = 0.4714, \frac{0.2000}{0.2121} = 0.9428]$$

Resulting normalized matrix (shape 4×4):

$\langle \text{bos} \rangle$	0.9998	0.9998	0.9998	0.9998
I	1.6329	0.8165	-0.8165	0.0000
love	-0.5164	1.5492	1.0328	0.5164
AI	1.4142	-0.9428	0.4714	0.9428

Step 4a: MLA down-projection into c_KV

W^{DKV} (shape 4×2):

$$\begin{bmatrix} 0.3000 & 0.1000 \\ 0.2000 & -0.2000 \\ 0.1000 & 0.3000 \\ 0.0000 & 0.2000 \end{bmatrix}$$

Xn (**every token as a row**) (shape 4×4):

$\langle \text{bos} \rangle$	0.9998	0.9998	0.9998	0.9998
I	1.6329	0.8165	-0.8165	0.0000
love	-0.5164	1.5492	1.0328	0.5164
AI	1.4142	-0.9428	0.4714	0.9428

Expanded arithmetic, $c_{KV} = Xn \cdot W^{DKV}$, every token:

$\langle \text{bos} \rangle$ (Xn row = $[0.9998, 0.9998, 0.9998, 0.9998]$)

$$c_{KV}[0] = (0.9998 \times 0.3000) + (0.9998 \times 0.2000) + (0.9998 \times 0.1000) + (0.9998 \times 0.0000) = 0.5999$$

$$c_{KV}[1] = (0.9998 \times 0.1000) + (0.9998 \times -0.2000) + (0.9998 \times 0.3000) + (0.9998 \times 0.2000) = 0.3999$$

I (Xn row = $[1.6329, 0.8165, -0.8165, 0.0000]$)

$$c_{KV}[0] = (1.6329 \times 0.3000) + (0.8165 \times 0.2000) + (-0.8165 \times 0.1000) + (0.0000 \times 0.0000) = 0.5715$$

$$c_{KV}[1] = (1.6329 \times 0.1000) + (0.8165 \times -0.2000) + (-0.8165 \times 0.3000) + (0.0000 \times 0.2000) = -0.2449$$

love (*Xn* row = [-0.5164, 1.5492, 1.0328, 0.5164])

$$c_{KV}[0] = (-0.5164 \times 0.3000) + (1.5492 \times 0.2000) + (1.0328 \times 0.1000) + (0.5164 \times 0.0000) = 0.2582$$

$$c_{KV}[1] = (-0.5164 \times 0.1000) + (1.5492 \times -0.2000) + (1.0328 \times 0.3000) + (0.5164 \times 0.2000) = 0.0516$$

AI (*Xn* row = [1.4142, -0.9428, 0.4714, 0.9428])

$$c_{KV}[0] = (1.4142 \times 0.3000) + (-0.9428 \times 0.2000) + (0.4714 \times 0.1000) + (0.9428 \times 0.0000) = 0.2828$$

$$c_{KV}[1] = (1.4142 \times 0.1000) + (-0.9428 \times -0.2000) + (0.4714 \times 0.3000) + (0.9428 \times 0.2000) = 0.6600$$

Resulting c_{KV} matrix (shape 4×2):

<i><bos></i>	0.5999	0.3999
I	0.5715	-0.2449
love	0.2582	0.0516
AI	0.2828	0.6600

Step 4b: MLA down-projection into c_Q

W^{DQ} (shape 4×2):

$$\begin{bmatrix} 0.2000 & 0.0000 \\ 0.1000 & 0.3000 \\ 0.0000 & 0.2000 \\ 0.3000 & 0.1000 \end{bmatrix}$$

Xn (**every token as a row**) (shape 4×4):

<i><bos></i>	0.9998	0.9998	0.9998	0.9998
I	1.6329	0.8165	-0.8165	0.0000
love	-0.5164	1.5492	1.0328	0.5164
AI	1.4142	-0.9428	0.4714	0.9428

Expanded arithmetic, $c_Q = Xn \cdot W^{DQ}$, every token:

<bos> (*Xn* row = [0.9998, 0.9998, 0.9998, 0.9998])

$$c_Q[0] = (0.9998 \times 0.2000) + (0.9998 \times 0.1000) + (0.9998 \times 0.0000) + (0.9998 \times 0.3000) = 0.5999$$

$$c_Q[1] = (0.9998 \times 0.0000) + (0.9998 \times 0.3000) + (0.9998 \times 0.2000) + (0.9998 \times 0.1000) = 0.5999$$

I (*Xn* row = [1.6329, 0.8165, -0.8165, 0.0000])

$$c_Q[0] = (1.6329 \times 0.2000) + (0.8165 \times 0.1000) + (-0.8165 \times 0.0000) + (0.0000 \times 0.3000) = 0.4082$$

$$c_Q[1] = (1.6329 \times 0.0000) + (0.8165 \times 0.3000) + (-0.8165 \times 0.2000) + (0.0000 \times 0.1000) = 0.0816$$

love (Xn row = $[-0.5164, 1.5492, 1.0328, 0.5164]$)

$$c_Q[0] = (-0.5164 \times 0.2000) + (1.5492 \times 0.1000) + (1.0328 \times 0.0000) + (0.5164 \times 0.3000) = 0.2066$$

$$c_Q[1] = (-0.5164 \times 0.0000) + (1.5492 \times 0.3000) + (1.0328 \times 0.2000) + (0.5164 \times 0.1000) = 0.7229$$

AI (Xn row = $[1.4142, -0.9428, 0.4714, 0.9428]$)

$$c_Q[0] = (1.4142 \times 0.2000) + (-0.9428 \times 0.1000) + (0.4714 \times 0.0000) + (0.9428 \times 0.3000) = 0.4714$$

$$c_Q[1] = (1.4142 \times 0.0000) + (-0.9428 \times 0.3000) + (0.4714 \times 0.2000) + (0.9428 \times 0.1000) = -0.0943$$

Resulting c_Q matrix (shape 4×2):

$\langle \text{bos} \rangle$	0.5999	0.5999
I	0.4082	0.0816
love	0.2066	0.7229
AI	0.4714	-0.0943

Step 5 (Head 1): up-project into $K_content$

W_1^{UK} (shape 2×1):

$$\begin{bmatrix} 0.6000 \\ 0.3000 \end{bmatrix}$$

c_{KV} (**every token as a row**) (shape 4×2):

$\langle \text{bos} \rangle$	0.5999	0.3999
I	0.5715	-0.2449
love	0.2582	0.0516
AI	0.2828	0.6600

Expanded arithmetic, $K_{content} = c_{KV} \cdot W_1^{UK}$, every token:

$\langle \text{bos} \rangle$ (c_{KV} row = $[0.5999, 0.3999]$)

$$K_{content}[0] = (0.5999 \times 0.6000) + (0.3999 \times 0.3000) = 0.4799$$

I (c_{KV} row = $[0.5715, -0.2449]$)

$$K_{content}[0] = (0.5715 \times 0.6000) + (-0.2449 \times 0.3000) = 0.2694$$

love (c_{KV} row = $[0.2582, 0.0516]$)

$$K_{content}[0] = (0.2582 \times 0.6000) + (0.0516 \times 0.3000) = 0.1704$$

AI (c_{KV} row = $[0.2828, 0.6600]$)

$$K_{content}[0] = (0.2828 \times 0.6000) + (0.6600 \times 0.3000) = 0.3677$$

Resulting $K_{content}$ matrix (shape 4×1):

<bos>	0.4799
I	0.2694
love	0.1704
AI	0.3677

Step 5 (Head 1): up-project into $Q_content$

W_1^{UQ} (shape 2×1):

$$\begin{bmatrix} 0.5000 \\ 0.4000 \end{bmatrix}$$

c_Q (every token as a row) (shape 4×2):

<bos>	0.5999	0.5999
I	0.4082	0.0816
love	0.2066	0.7229
AI	0.4714	-0.0943

Expanded arithmetic, $Q_{content} = c_Q \cdot W_1^{UQ}$, every token:

<bos> (c_Q row = $[0.5999, 0.5999]$)

$$Q_{content}[0] = (0.5999 \times 0.5000) + (0.5999 \times 0.4000) = 0.5399$$

I (c_Q row = $[0.4082, 0.0816]$)

$$Q_{content}[1] = (0.4082 \times 0.5000) + (0.0816 \times 0.4000) = 0.2368$$

love (c_Q row = $[0.2066, 0.7229]$)

$$Q_{content}[2] = (0.2066 \times 0.5000) + (0.7229 \times 0.4000) = 0.3925$$

AI (c_Q row = $[0.4714, -0.0943]$)

$$Q_{content}[3] = (0.4714 \times 0.5000) + (-0.0943 \times 0.4000) = 0.1980$$

Resulting $Q_{content}$ matrix (shape 4×1):

<bos>	0.5399
I	0.2368
love	0.3925
AI	0.1980

Step 5 (Head 1): up-project into V W_1^{UV} (shape 2×1):

$$\begin{bmatrix} 0.4000 \\ 0.2000 \end{bmatrix}$$

 c_{KV} (every token as a row) (shape 4×2):

<bos>	0.5999	0.3999
I	0.5715	-0.2449
love	0.2582	0.0516
AI	0.2828	0.6600

Expanded arithmetic, $V = c_{KV} \cdot W_1^{UV}$, every token:<bos> (c_{KV} row = $[0.5999, 0.3999]$)

$$V[0] = (0.5999 \times 0.4000) + (0.3999 \times 0.2000) = 0.3199$$

I (c_{KV} row = $[0.5715, -0.2449]$)

$$V[0] = (0.5715 \times 0.4000) + (-0.2449 \times 0.2000) = 0.1796$$

love (c_{KV} row = $[0.2582, 0.0516]$)

$$V[0] = (0.2582 \times 0.4000) + (0.0516 \times 0.2000) = 0.1136$$

AI (c_{KV} row = $[0.2828, 0.6600]$)

$$V[0] = (0.2828 \times 0.4000) + (0.6600 \times 0.2000) = 0.2451$$

Resulting V matrix (shape 4×1):

<bos>	0.3199
I	0.1796
love	0.1136
AI	0.2451

Step 5 (Head 2): up-project into K_content W_2^{UK} (shape 2×1):

$$\begin{bmatrix} 0.2000 \\ 0.7000 \end{bmatrix}$$

 c_{KV} (every token as a row) (shape 4×2):

<bos>	0.5999	0.3999
I	0.5715	-0.2449
love	0.2582	0.0516
AI	0.2828	0.6600

Expanded arithmetic, $K_{content} = c_{KV} \cdot W_2^{UK}$, every token:

$\langle bos \rangle$ (c_{KV} row = [0.5999, 0.3999])

$$K_{content}[0] = (0.5999 \times 0.2000) + (0.3999 \times 0.7000) = 0.3999$$

I (c_{KV} row = [0.5715, -0.2449])

$$K_{content}[0] = (0.5715 \times 0.2000) + (-0.2449 \times 0.7000) = -0.0572$$

$love$ (c_{KV} row = [0.2582, 0.0516])

$$K_{content}[0] = (0.2582 \times 0.2000) + (0.0516 \times 0.7000) = 0.0878$$

AI (c_{KV} row = [0.2828, 0.6600])

$$K_{content}[0] = (0.2828 \times 0.2000) + (0.6600 \times 0.7000) = 0.5185$$

Resulting $K_{content}$ matrix (shape 4×1):

$$\begin{array}{c|c} \langle bos \rangle & 0.3999 \\ I & -0.0572 \\ love & 0.0878 \\ AI & 0.5185 \end{array}$$

Step 5 (Head 2): up-project into $Q_{content}$

W_2^{UQ} (shape 2×1):

$$\begin{bmatrix} 0.3000 \\ 0.6000 \end{bmatrix}$$

c_Q (every token as a row) (shape 4×2):

$$\begin{array}{c|cc} \langle bos \rangle & 0.5999 & 0.5999 \\ I & 0.4082 & 0.0816 \\ love & 0.2066 & 0.7229 \\ AI & 0.4714 & -0.0943 \end{array}$$

Expanded arithmetic, $Q_{content} = c_Q \cdot W_2^{UQ}$, every token:

$\langle bos \rangle$ (c_Q row = [0.5999, 0.5999])

$$Q_{content}[0] = (0.5999 \times 0.3000) + (0.5999 \times 0.6000) = 0.5399$$

I (c_Q row = [0.4082, 0.0816])

$$Q_{content}[0] = (0.4082 \times 0.3000) + (0.0816 \times 0.6000) = 0.1715$$

$love$ (c_Q row = [0.2066, 0.7229])

$$Q_{content}[0] = (0.2066 \times 0.3000) + (0.7229 \times 0.6000) = 0.4957$$

$$AI \text{ } (c_Q \text{ row} = [0.4714, -0.0943])$$

$$Q_{content}[0] = (0.4714 \times 0.3000) + (-0.0943 \times 0.6000) = 0.0849$$

Resulting $Q_{content}$ matrix (shape 4×1):

$\langle \text{bos} \rangle$	0.5399
I	0.1715
love	0.4957
AI	0.0849

Step 5 (Head 2): up-project into V

$$W_2^{UV} \text{ (shape } 2 \times 1\text{):}$$

$$\begin{bmatrix} 0.1000 \\ 0.5000 \end{bmatrix}$$

$$c_{KV} \text{ (every token as a row) (shape } 4 \times 2\text{):}$$

$\langle \text{bos} \rangle$	0.5999	0.3999
I	0.5715	-0.2449
love	0.2582	0.0516
AI	0.2828	0.6600

Expanded arithmetic, $V = c_{KV} \cdot W_2^{UV}$, every token:

$$\langle \text{bos} \rangle \text{ } (c_{KV} \text{ row} = [0.5999, 0.3999])$$

$$V[0] = (0.5999 \times 0.1000) + (0.3999 \times 0.5000) = 0.2599$$

$$I \text{ } (c_{KV} \text{ row} = [0.5715, -0.2449])$$

$$V[0] = (0.5715 \times 0.1000) + (-0.2449 \times 0.5000) = -0.0653$$

$$\text{love} \text{ } (c_{KV} \text{ row} = [0.2582, 0.0516])$$

$$V[0] = (0.2582 \times 0.1000) + (0.0516 \times 0.5000) = 0.0516$$

$$AI \text{ } (c_{KV} \text{ row} = [0.2828, 0.6600])$$

$$V[0] = (0.2828 \times 0.1000) + (0.6600 \times 0.5000) = 0.3583$$

Resulting V matrix (shape 4×1):

$\langle \text{bos} \rangle$	0.2599
I	-0.0653
love	0.0516
AI	0.3583

Step 6a: Project X_n into Q_{rope} (BEFORE rotation)

W^{QR} (shape 4×2):

$$\begin{bmatrix} 0.4000 & 0.1000 \\ 0.2000 & 0.3000 \\ 0.1000 & 0.2000 \\ 0.0000 & 0.4000 \end{bmatrix}$$

X_n (every token as a row) (shape 4×4):

$$\begin{array}{c|cccc} \langle \text{bos} \rangle & 0.9998 & 0.9998 & 0.9998 & 0.9998 \\ \text{I} & 1.6329 & 0.8165 & -0.8165 & 0.0000 \\ \text{love} & -0.5164 & 1.5492 & 1.0328 & 0.5164 \\ \text{AI} & 1.4142 & -0.9428 & 0.4714 & 0.9428 \end{array}$$

Expanded arithmetic, $Q_{rope} = X_n \cdot W^{QR}$, every token:

$\langle \text{bos} \rangle$ (X_n row = $[0.9998, 0.9998, 0.9998, 0.9998]$)

$$Q_{rope}[0] = (0.9998 \times 0.4000) + (0.9998 \times 0.2000) + (0.9998 \times 0.1000) + (0.9998 \times 0.0000) = 0.6999$$

$$Q_{rope}[1] = (0.9998 \times 0.1000) + (0.9998 \times 0.3000) + (0.9998 \times 0.2000) + (0.9998 \times 0.4000) = 0.9998$$

I (X_n row = $[1.6329, 0.8165, -0.8165, 0.0000]$)

$$Q_{rope}[0] = (1.6329 \times 0.4000) + (0.8165 \times 0.2000) + (-0.8165 \times 0.1000) + (0.0000 \times 0.0000) = 0.7348$$

$$Q_{rope}[1] = (1.6329 \times 0.1000) + (0.8165 \times 0.3000) + (-0.8165 \times 0.2000) + (0.0000 \times 0.4000) = 0.2449$$

love (X_n row = $[-0.5164, 1.5492, 1.0328, 0.5164]$)

$$Q_{rope}[0] = (-0.5164 \times 0.4000) + (1.5492 \times 0.2000) + (1.0328 \times 0.1000) + (0.5164 \times 0.0000) = 0.2066$$

$$Q_{rope}[1] = (-0.5164 \times 0.1000) + (1.5492 \times 0.3000) + (1.0328 \times 0.2000) + (0.5164 \times 0.4000) = 0.8262$$

AI (X_n row = $[1.4142, -0.9428, 0.4714, 0.9428]$)

$$Q_{rope}[0] = (1.4142 \times 0.4000) + (-0.9428 \times 0.2000) + (0.4714 \times 0.1000) + (0.9428 \times 0.0000) = 0.4243$$

$$Q_{rope}[1] = (1.4142 \times 0.1000) + (-0.9428 \times 0.3000) + (0.4714 \times 0.2000) + (0.9428 \times 0.4000) = 0.3300$$

Resulting Q_{rope} matrix (shape 4×2):

$$\begin{array}{c|cc} \langle \text{bos} \rangle & 0.6999 & 0.9998 \\ \text{I} & 0.7348 & 0.2449 \\ \text{love} & 0.2066 & 0.8262 \\ \text{AI} & 0.4243 & 0.3300 \end{array}$$

Step 6b: Project X_n into K_{rope} (BEFORE rotation, shared across heads)

W^{KR} (shape 4×2):

$$\begin{bmatrix} 0.3000 & 0.2000 \\ 0.1000 & 0.4000 \\ 0.2000 & 0.1000 \\ 0.3000 & 0.0000 \end{bmatrix}$$

X_n (every token as a row) (shape 4×4):

$$\begin{array}{c|cccc} <bos> & 0.9998 & 0.9998 & 0.9998 & 0.9998 \\ I & 1.6329 & 0.8165 & -0.8165 & 0.0000 \\ love & -0.5164 & 1.5492 & 1.0328 & 0.5164 \\ AI & 1.4142 & -0.9428 & 0.4714 & 0.9428 \end{array}$$

Expanded arithmetic, $K_{rope} = X_n \cdot W^{KR}$, every token:

$<bos>$ (X_n row = $[0.9998, 0.9998, 0.9998, 0.9998]$)

$$K_{rope}[0] = (0.9998 \times 0.3000) + (0.9998 \times 0.1000) + (0.9998 \times 0.2000) + (0.9998 \times 0.3000) = 0.8998$$

$$K_{rope}[1] = (0.9998 \times 0.2000) + (0.9998 \times 0.4000) + (0.9998 \times 0.1000) + (0.9998 \times 0.0000) = 0.6999$$

I (X_n row = $[1.6329, 0.8165, -0.8165, 0.0000]$)

$$K_{rope}[0] = (1.6329 \times 0.3000) + (0.8165 \times 0.1000) + (-0.8165 \times 0.2000) + (0.0000 \times 0.3000) = 0.4082$$

$$K_{rope}[1] = (1.6329 \times 0.2000) + (0.8165 \times 0.4000) + (-0.8165 \times 0.1000) + (0.0000 \times 0.0000) = 0.5715$$

$love$ (X_n row = $[-0.5164, 1.5492, 1.0328, 0.5164]$)

$$K_{rope}[0] = (-0.5164 \times 0.3000) + (1.5492 \times 0.1000) + (1.0328 \times 0.2000) + (0.5164 \times 0.3000) = 0.3615$$

$$K_{rope}[1] = (-0.5164 \times 0.2000) + (1.5492 \times 0.4000) + (1.0328 \times 0.1000) + (0.5164 \times 0.0000) = 0.6197$$

AI (X_n row = $[1.4142, -0.9428, 0.4714, 0.9428]$)

$$K_{rope}[0] = (1.4142 \times 0.3000) + (-0.9428 \times 0.1000) + (0.4714 \times 0.2000) + (0.9428 \times 0.3000) = 0.7071$$

$$K_{rope}[1] = (1.4142 \times 0.2000) + (-0.9428 \times 0.4000) + (0.4714 \times 0.1000) + (0.9428 \times 0.0000) = -0.0471$$

Resulting K_{rope} matrix (shape 4×2):

$$\begin{array}{c|cc} <bos> & 0.8998 & 0.6999 \\ I & 0.4082 & 0.5715 \\ love & 0.3615 & 0.6197 \\ AI & 0.7071 & -0.0471 \end{array}$$

Step 7a: Apply RoPE rotation to Q_rope

Input matrix BEFORE rotation (shape 4×2):

$$\begin{array}{c|cc} \langle \text{bos} \rangle & 0.6999 & 0.9998 \\ I & 0.7348 & 0.2449 \\ \text{love} & 0.2066 & 0.8262 \\ AI & 0.4243 & 0.3300 \end{array}$$

$$R(m) = \begin{bmatrix} \cos(m) & -\sin(m) \\ \sin(m) & \cos(m) \end{bmatrix} \quad (\text{angle} = m \times \theta, \theta = 1 \text{ here})$$

$\langle \text{bos} \rangle$ (pos=0): angle = $0 \times 1.0 = 0.0000$ rad

$$\text{rotated}_0 = \cos(0.0000) \times 0.6999 - \sin(0.0000) \times 0.9998 = 0.6999 - 0.0000 = 0.6999$$

$$\text{rotated}_1 = \sin(0.0000) \times 0.6999 + \cos(0.0000) \times 0.9998 = 0.0000 + 0.9998 = 0.9998$$

I (pos=1): angle = $1 \times 1.0 = 1.0000$ rad

$$\text{rotated}_0 = \cos(1.0000) \times 0.7348 - \sin(1.0000) \times 0.2449 = 0.3970 - 0.2061 = 0.1909$$

$$\text{rotated}_1 = \sin(1.0000) \times 0.7348 + \cos(1.0000) \times 0.2449 = 0.6183 + 0.1323 = 0.7507$$

love (pos=2): angle = $2 \times 1.0 = 2.0000$ rad

$$\text{rotated}_0 = \cos(2.0000) \times 0.2066 - \sin(2.0000) \times 0.8262 = -0.0860 - 0.7513 = -0.8372$$

$$\text{rotated}_1 = \sin(2.0000) \times 0.2066 + \cos(2.0000) \times 0.8262 = 0.1878 + -0.3438 = -0.1560$$

AI (pos=3): angle = $3 \times 1.0 = 3.0000$ rad

$$\text{rotated}_0 = \cos(3.0000) \times 0.4243 - \sin(3.0000) \times 0.3300 = -0.4200 - 0.0466 = -0.4666$$

$$\text{rotated}_1 = \sin(3.0000) \times 0.4243 + \cos(3.0000) \times 0.3300 = 0.0599 + -0.3267 = -0.2668$$

Resulting rotated matrix (shape 4×2):

$$\begin{array}{c|cc} \langle \text{bos} \rangle & 0.6999 & 0.9998 \\ I & 0.1909 & 0.7507 \\ \text{love} & -0.8372 & -0.1560 \\ AI & -0.4666 & -0.2668 \end{array}$$

Step 7b: Apply RoPE rotation to K_rope

Input matrix BEFORE rotation (shape 4×2):

$$\begin{array}{c|cc} \langle \text{bos} \rangle & 0.8998 & 0.6999 \\ I & 0.4082 & 0.5715 \\ \text{love} & 0.3615 & 0.6197 \\ AI & 0.7071 & -0.0471 \end{array}$$

$$R(m) = \begin{bmatrix} \cos(m) & -\sin(m) \\ \sin(m) & \cos(m) \end{bmatrix} \quad (\text{angle} = m \times \theta, \theta = 1 \text{ here})$$

$\langle \text{bos} \rangle$ (pos=0): angle= $0 \times 1.0 = 0.0000$ rad

$$\text{rotated}_0 = \cos(0.0000) \times 0.8998 - \sin(0.0000) \times 0.6999 = 0.8998 - 0.0000 = 0.8998$$

$$\text{rotated}_1 = \sin(0.0000) \times 0.8998 + \cos(0.0000) \times 0.6999 = 0.0000 + 0.6999 = 0.6999$$

I (pos=1): angle= $1 \times 1.0 = 1.0000$ rad

$$\text{rotated}_0 = \cos(1.0000) \times 0.4082 - \sin(1.0000) \times 0.5715 = 0.2206 - 0.4809 = -0.2604$$

$$\text{rotated}_1 = \sin(1.0000) \times 0.4082 + \cos(1.0000) \times 0.5715 = 0.3435 + 0.3088 = 0.6523$$

love (pos=2): angle= $2 \times 1.0 = 2.0000$ rad

$$\text{rotated}_0 = \cos(2.0000) \times 0.3615 - \sin(2.0000) \times 0.6197 = -0.1504 - 0.5635 = -0.7139$$

$$\text{rotated}_1 = \sin(2.0000) \times 0.3615 + \cos(2.0000) \times 0.6197 = 0.3287 + -0.2579 = 0.0708$$

AI (pos=3): angle= $3 \times 1.0 = 3.0000$ rad

$$\text{rotated}_0 = \cos(3.0000) \times 0.7071 - \sin(3.0000) \times -0.0471 = -0.7000 - -0.0067 = -0.6934$$

$$\text{rotated}_1 = \sin(3.0000) \times 0.7071 + \cos(3.0000) \times -0.0471 = 0.0998 + 0.0467 = 0.1465$$

Resulting rotated matrix (shape 4×2):

$\langle \text{bos} \rangle$	0.8998	0.6999
I	-0.2604	0.6523
love	-0.7139	0.0708
AI	-0.6934	0.1465

Step 8: Assemble full per-head Q and K = [content ; rotated rope]

Head 1 full Q = [Q_content | Q_rope_rotated] (shape 4×3):

$\langle \text{bos} \rangle$	0.5399	0.6999	0.9998
I	0.2368	0.1909	0.7507
love	0.3925	-0.8372	-0.1560
AI	0.1980	-0.4666	-0.2668

Head 1 full K = [K_content | K_rope_rotated] (shape 4×3):

$\langle \text{bos} \rangle$	0.4799	0.8998	0.6999
I	0.2694	-0.2604	0.6523
love	0.1704	-0.7139	0.0708
AI	0.3677	-0.6934	0.1465

Head 2 full Q = [Q_content | Q_rope_rotated] (shape 4×3):

$\langle \text{bos} \rangle$	0.5399	0.6999	0.9998
I	0.1715	0.1909	0.7507
love	0.4957	-0.8372	-0.1560
AI	0.0849	-0.4666	-0.2668

Head 2 full K = [K_content | K_rope_rotated] (shape 4×3):

<bos>	0.3999	0.8998	0.6999
I	-0.0572	-0.2604	0.6523
love	0.0878	-0.7139	0.0708
AI	0.5185	-0.6934	0.1465

Step 9 (Head 1): full causal attention

Head 1: Q matrix (shape 4×3):

<bos>	0.5399	0.6999	0.9998
I	0.2368	0.1909	0.7507
love	0.3925	-0.8372	-0.1560
AI	0.1980	-0.4666	-0.2668

Head 1: K matrix (shape 4×3):

<bos>	0.4799	0.8998	0.6999
I	0.2694	-0.2604	0.6523
love	0.1704	-0.7139	0.0708
AI	0.3677	-0.6934	0.1465

Head 1: V matrix (shape 4×1):

<bos>	0.3199
I	0.1796
love	0.1136
AI	0.2451

Causal attention scores $\text{score}(i, j) = (Q_i \cdot K_j) / \sqrt{3} = (Q_i \cdot K_j) / 1.7321$, only for $j \leq i$:

Query: <bos> (position 0, sees positions 0..0)

$$\text{score}(\text{<bos>, <bos>}) = \frac{(0.5399 \times 0.4799) + (0.6999 \times 0.8998) + (0.9998 \times 0.6999)}{1.7321} = \frac{1.5886}{1.7321} = 0.9172$$

Query: I (position 1, sees positions 0..1)

$$\text{score}(\text{I, <bos>}) = \frac{(0.2368 \times 0.4799) + (0.1909 \times 0.8998) + (0.7507 \times 0.6999)}{1.7321} = \frac{0.8108}{1.7321} = 0.4681$$

$$\text{score}(\text{I, I}) = \frac{(0.2368 \times 0.2694) + (0.1909 \times -0.2604) + (0.7507 \times 0.6523)}{1.7321} = \frac{0.5038}{1.7321} = 0.2908$$

Query: love (position 2, sees positions 0..2)

$$\text{score}(\text{love, <bos>}) = \frac{(0.3925 \times 0.4799) + (-0.8372 \times 0.8998) + (-0.1560 \times 0.6999)}{1.7321} = \frac{-0.6742}{1.7321} = -0.3893$$

$$\text{score}(\text{love, I}) = \frac{(0.3925 \times 0.2694) + (-0.8372 \times -0.2604) + (-0.1560 \times 0.6523)}{1.7321} = \frac{0.2220}{1.7321} = 0.1281$$

$$\text{score}(\text{love}, \text{love}) = \frac{(0.3925 \times 0.1704) + (-0.8372 \times -0.7139) + (-0.1560 \times 0.0708)}{1.7321} = \frac{0.6535}{1.7321} = 0.3773$$

Query: AI (position 3, sees positions 0..3)

$$\text{score}(\text{AI}, <\text{bos}>) = \frac{(0.1980 \times 0.4799) + (-0.4666 \times 0.8998) + (-0.2668 \times 0.6999)}{1.7321} = \frac{-0.5115}{1.7321} = -0.2953$$

$$\text{score}(\text{AI}, \text{I}) = \frac{(0.1980 \times 0.2694) + (-0.4666 \times -0.2604) + (-0.2668 \times 0.6523)}{1.7321} = \frac{0.0008}{1.7321} = 0.0005$$

$$\text{score}(\text{AI}, \text{love}) = \frac{(0.1980 \times 0.1704) + (-0.4666 \times -0.7139) + (-0.2668 \times 0.0708)}{1.7321} = \frac{0.3479}{1.7321} = 0.2009$$

$$\text{score}(\text{AI}, \text{AI}) = \frac{(0.1980 \times 0.3677) + (-0.4666 \times -0.6934) + (-0.2668 \times 0.1465)}{1.7321} = \frac{0.3572}{1.7321} = 0.2063$$

Softmax over valid (unmasked) positions, every row:

<bos>: scores = [0.9172]

$$\text{exponentials} = [e^{0.9172} = 1.0000]$$

$$\text{sum} = 1.0000$$

$$\text{weights} = [\frac{1.0000}{1.0000} = 1.0000]$$

I: scores = [0.4681, 0.2908]

$$\text{exponentials} = [e^{0.4681} = 1.0000, e^{0.2908} = 0.8376]$$

$$\text{sum} = 1.8376$$

$$\text{weights} = [\frac{1.0000}{1.8376} = 0.5442, \frac{0.8376}{1.8376} = 0.4558]$$

love: scores = [-0.3893, 0.1281, 0.3773]

$$\text{exponentials} = [e^{-0.3893} = 0.4646, e^{0.1281} = 0.7794, e^{0.3773} = 1.0000]$$

$$\text{sum} = 2.2441$$

$$\text{weights} = [\frac{0.4646}{2.2441} = 0.2070, \frac{0.7794}{2.2441} = 0.3473, \frac{1.0000}{2.2441} = 0.4456]$$

AI: scores = [-0.2953, 0.0005, 0.2009, 0.2063]

$$\text{exponentials} = [e^{-0.2953} = 0.6056, e^{0.0005} = 0.8140, e^{0.2009} = 0.9946, e^{0.2063} = 1.0000]$$

$$\text{sum} = 3.4142$$

$$\text{weights} = [\frac{0.6056}{3.4142} = 0.1774, \frac{0.8140}{3.4142} = 0.2384, \frac{0.9946}{3.4142} = 0.2913, \frac{1.0000}{3.4142} = 0.2929]$$

Attention output = weighted sum of V, every token:

$$<\text{bos}>_{[0]} = (1.0000 \times 0.3199) = 0.3199$$

$$I_{[0]} = (0.5442 \times 0.3199) + (0.4558 \times 0.1796) = 0.2560$$

$$\text{love}_{[0]} = (0.2070 \times 0.3199) + (0.3473 \times 0.1796) + (0.4456 \times 0.1136) = 0.1793$$

$$AI_{[0]} = (0.1774 \times 0.3199) + (0.2384 \times 0.1796) + (0.2913 \times 0.1136) + (0.2929 \times 0.2451) = 0.2045$$

Head 1: resulting attention-output matrix (shape 4×1):

$\langle \text{bos} \rangle$	0.3199
I	0.2560
love	0.1793
AI	0.2045

Step 9 (Head 2): full causal attention

Head 2: Q matrix (shape 4×3):

$\langle \text{bos} \rangle$	0.5399	0.6999	0.9998
I	0.1715	0.1909	0.7507
love	0.4957	-0.8372	-0.1560
AI	0.0849	-0.4666	-0.2668

Head 2: K matrix (shape 4×3):

$\langle \text{bos} \rangle$	0.3999	0.8998	0.6999
I	-0.0572	-0.2604	0.6523
love	0.0878	-0.7139	0.0708
AI	0.5185	-0.6934	0.1465

Head 2: V matrix (shape 4×1):

$\langle \text{bos} \rangle$	0.2599
I	-0.0653
love	0.0516
AI	0.3583

Causal attention scores $\text{score}(i, j) = (Q_i \cdot K_j) / \sqrt{3} = (Q_i \cdot K_j) / 1.7321$, only for $j \leq i$:

Query: $\langle \text{bos} \rangle$ (position 0, sees positions 0..0)

$$\text{score}(\langle \text{bos} \rangle, \langle \text{bos} \rangle) = \frac{(0.5399 \times 0.3999) + (0.6999 \times 0.8998) + (0.9998 \times 0.6999)}{1.7321} = \frac{1.5454}{1.7321} = 0.8922$$

Query: I (position 1, sees positions 0..1)

$$\text{score}(I, \langle \text{bos} \rangle) = \frac{(0.1715 \times 0.3999) + (0.1909 \times 0.8998) + (0.7507 \times 0.6999)}{1.7321} = \frac{0.7657}{1.7321} = 0.4421$$

$$\text{score}(I, I) = \frac{(0.1715 \times -0.0572) + (0.1909 \times -0.2604) + (0.7507 \times 0.6523)}{1.7321} = \frac{0.4302}{1.7321} = 0.2484$$

Query: love (position 2, sees positions 0..2)

$$\text{score}(\text{love}, \langle \text{bos} \rangle) = \frac{(0.4957 \times 0.3999) + (-0.8372 \times 0.8998) + (-0.1560 \times 0.6999)}{1.7321} = \frac{-0.6643}{1.7321} = -0.3835$$

$$\text{score}(\text{love}, \text{I}) = \frac{(0.4957 \times -0.0572) + (-0.8372 \times -0.2604) + (-0.1560 \times 0.6523)}{1.7321} = \frac{0.0879}{1.7321} = 0.0507$$

$$\text{score}(\text{love}, \text{love}) = \frac{(0.4957 \times 0.0878) + (-0.8372 \times -0.7139) + (-0.1560 \times 0.0708)}{1.7321} = \frac{0.6302}{1.7321} = 0.3638$$

Query: AI (position 3, sees positions 0..3)

$$\text{score}(\text{AI}, \langle \text{bos} \rangle) = \frac{(0.0849 \times 0.3999) + (-0.4666 \times 0.8998) + (-0.2668 \times 0.6999)}{1.7321} = \frac{-0.5726}{1.7321} = -0.3306$$

$$\text{score}(\text{AI}, \text{I}) = \frac{(0.0849 \times -0.0572) + (-0.4666 \times -0.2604) + (-0.2668 \times 0.6523)}{1.7321} = \frac{-0.0574}{1.7321} = -0.0331$$

$$\text{score}(\text{AI}, \text{love}) = \frac{(0.0849 \times 0.0878) + (-0.4666 \times -0.7139) + (-0.2668 \times 0.0708)}{1.7321} = \frac{0.3216}{1.7321} = 0.1857$$

$$\text{score}(\text{AI}, \text{AI}) = \frac{(0.0849 \times 0.5185) + (-0.4666 \times -0.6934) + (-0.2668 \times 0.1465)}{1.7321} = \frac{0.3284}{1.7321} = 0.1896$$

Softmax over valid (unmasked) positions, every row:

<bos>: scores = [0.8922]

$$\text{exponentials} = [e^{0.8922} = 1.0000]$$

$$\text{sum} = 1.0000$$

$$\text{weights} = [\frac{1.0000}{1.0000} = 1.0000]$$

I: scores = [0.4421, 0.2484]

$$\text{exponentials} = [e^{0.4421} = 1.0000, e^{0.2484} = 0.8239]$$

$$\text{sum} = 1.8239$$

$$\text{weights} = [\frac{1.0000}{1.8239} = 0.5483, \frac{0.8239}{1.8239} = 0.4517]$$

love: scores = [-0.3835, 0.0507, 0.3638]

$$\text{exponentials} = [e^{-0.3835} = 0.4736, e^{0.0507} = 0.7312, e^{0.3638} = 1.0000]$$

$$\text{sum} = 2.2048$$

$$\text{weights} = [\frac{0.4736}{2.2048} = 0.2148, \frac{0.7312}{2.2048} = 0.3316, \frac{1.0000}{2.2048} = 0.4536]$$

AI: scores = [-0.3306, -0.0331, 0.1857, 0.1896]

$$\text{exponentials} = [e^{-0.3306} = 0.5944, e^{-0.0331} = 0.8003, e^{0.1857} = 0.9961, e^{0.1896} = 1.0000]$$

$$\text{sum} = 3.3908$$

$$\text{weights} = [\frac{0.5944}{3.3908} = 0.1753, \frac{0.8003}{3.3908} = 0.2360, \frac{0.9961}{3.3908} = 0.2938, \frac{1.0000}{3.3908} = 0.2949]$$

Attention output = weighted sum of V, every token:

$$\langle \text{bos} \rangle_{[0]} = (1.0000 \times 0.2599) = 0.2599$$

$$I_{[0]} = (0.5483 \times 0.2599) + (0.4517 \times -0.0653) = 0.1130$$

$$\text{love}_{[0]} = (0.2148 \times 0.2599) + (0.3316 \times -0.0653) + (0.4536 \times 0.0516) = 0.0576$$

$$AI_{[0]} = (0.1753 \times 0.2599) + (0.2360 \times -0.0653) + (0.2938 \times 0.0516) + (0.2949 \times 0.3583) = 0.1510$$

Head 2: resulting attention-output matrix (shape 4×1):

$$\begin{array}{c|c} \langle \text{bos} \rangle & 0.2599 \\ I & 0.1130 \\ \text{love} & 0.0576 \\ AI & 0.1510 \end{array}$$

Step 10: Concat heads, project through W_o

Concat(head1, head2) (shape 4×2):

$$\begin{array}{c|cc} \langle \text{bos} \rangle & 0.3199 & 0.2599 \\ I & 0.2560 & 0.1130 \\ \text{love} & 0.1793 & 0.0576 \\ AI & 0.2045 & 0.1510 \end{array}$$

W_o (shape 2×4):

$$\begin{bmatrix} 0.5000 & 0.2000 & 0.1000 & 0.3000 \\ 0.3000 & 0.4000 & 0.2000 & 0.1000 \end{bmatrix}$$

concat (**every token as a row**) (shape 4×2):

$$\begin{array}{c|cc} \langle \text{bos} \rangle & 0.3199 & 0.2599 \\ I & 0.2560 & 0.1130 \\ \text{love} & 0.1793 & 0.0576 \\ AI & 0.2045 & 0.1510 \end{array}$$

Expanded arithmetic, $MultiHead = concat \cdot W_o$, every token:

$$\langle \text{bos} \rangle \text{ (concat row} = [0.3199, 0.2599])$$

$$MultiHead[0] = (0.3199 \times 0.5000) + (0.2599 \times 0.3000) = 0.2380$$

$$MultiHead[1] = (0.3199 \times 0.2000) + (0.2599 \times 0.4000) = 0.1680$$

$$MultiHead[2] = (0.3199 \times 0.1000) + (0.2599 \times 0.2000) = 0.0840$$

$$MultiHead[3] = (0.3199 \times 0.3000) + (0.2599 \times 0.1000) = 0.1220$$

I ($concat$ row = [0.2560, 0.1130])

$$MultiHead[0] = (0.2560 \times 0.5000) + (0.1130 \times 0.3000) = 0.1619$$

$$MultiHead[1] = (0.2560 \times 0.2000) + (0.1130 \times 0.4000) = 0.0964$$

$$MultiHead[2] = (0.2560 \times 0.1000) + (0.1130 \times 0.2000) = 0.0482$$

$$MultiHead[3] = (0.2560 \times 0.3000) + (0.1130 \times 0.1000) = 0.0881$$

$love$ ($concat$ row = [0.1793, 0.0576])

$$MultiHead[0] = (0.1793 \times 0.5000) + (0.0576 \times 0.3000) = 0.1069$$

$$MultiHead[1] = (0.1793 \times 0.2000) + (0.0576 \times 0.4000) = 0.0589$$

$$MultiHead[2] = (0.1793 \times 0.1000) + (0.0576 \times 0.2000) = 0.0294$$

$$MultiHead[3] = (0.1793 \times 0.3000) + (0.0576 \times 0.1000) = 0.0595$$

AI ($concat$ row = [0.2045, 0.1510])

$$MultiHead[0] = (0.2045 \times 0.5000) + (0.1510 \times 0.3000) = 0.1475$$

$$MultiHead[1] = (0.2045 \times 0.2000) + (0.1510 \times 0.4000) = 0.1013$$

$$MultiHead[2] = (0.2045 \times 0.1000) + (0.1510 \times 0.2000) = 0.0506$$

$$MultiHead[3] = (0.2045 \times 0.3000) + (0.1510 \times 0.1000) = 0.0764$$

Resulting $MultiHead$ matrix (shape 4×4):

<bos>	0.2380	0.1680	0.0840	0.1220
I	0.1619	0.0964	0.0482	0.0881
love	0.1069	0.0589	0.0294	0.0595
AI	0.1475	0.1013	0.0506	0.0764

Step 11: Residual add $Y = X + MultiHead(X)$

X (original embedding, pre-normalization) (shape 4×4):

<bos>	0.0500	0.0500	0.0500	0.0500
I	0.2000	0.1000	-0.1000	0.0000
love	-0.1000	0.3000	0.2000	0.1000
AI	0.3000	-0.2000	0.1000	0.2000

MultiHead(X) (shape 4×4):

<bos>	0.2380	0.1680	0.0840	0.1220
I	0.1619	0.0964	0.0482	0.0881
love	0.1069	0.0589	0.0294	0.0595
AI	0.1475	0.1013	0.0506	0.0764

Resulting Y matrix (shape 4×4):

<bos>	0.2880	0.2180	0.1340	0.1720
I	0.3619	0.1964	-0.0518	0.0881
love	0.0069	0.3589	0.2294	0.1595
AI	0.4475	-0.0987	0.1506	0.2764

Step 12: Second RMSNorm on Y

Input matrix (every token as a row) (shape 4×4):

<bos>	0.2880	0.2180	0.1340	0.1720
I	0.3619	0.1964	-0.0518	0.0881
love	0.0069	0.3589	0.2294	0.1595
AI	0.4475	-0.0987	0.1506	0.2764

<bos>: $v = [0.2880, 0.2180, 0.1340, 0.1720]$

$$\text{mean}(v^2) = \frac{0.0829 + 0.0475 + 0.0180 + 0.0296}{4} = \frac{0.1780}{4} = 0.0445$$

$$\text{denom} = \sqrt{0.0445 + 10^{-6}} = 0.2109$$

$$\text{normalized} = [\frac{0.2880}{0.2109} = 1.3652, \frac{0.2180}{0.2109} = 1.0334, \frac{0.1340}{0.2109} = 0.6352, \frac{0.1720}{0.2109} = 0.8153]$$

I: $v = [0.3619, 0.1964, -0.0518, 0.0881]$

$$\text{mean}(v^2) = \frac{0.1310 + 0.0386 + 0.0027 + 0.0078}{4} = \frac{0.1800}{4} = 0.0450$$

$$\text{denom} = \sqrt{0.0450 + 10^{-6}} = 0.2121$$

$$\text{normalized} = [\frac{0.3619}{0.2121} = 1.7060, \frac{0.1964}{0.2121} = 0.9259, \frac{-0.0518}{0.2121} = -0.2442, \frac{0.0881}{0.2121} = 0.4153]$$

love: $v = [0.0069, 0.3589, 0.2294, 0.1595]$

$$\text{mean}(v^2) = \frac{0.0000 + 0.1288 + 0.0526 + 0.0255}{4} = \frac{0.2069}{4} = 0.0517$$

$$\text{denom} = \sqrt{0.0517 + 10^{-6}} = 0.2275$$

$$\text{normalized} = [\frac{0.0069}{0.2275} = 0.0304, \frac{0.3589}{0.2275} = 1.5778, \frac{0.2294}{0.2275} = 1.0087, \frac{0.1595}{0.2275} = 0.7014]$$

AI: $v = [0.4475, -0.0987, 0.1506, 0.2764]$

$$\text{mean}(v^2) = \frac{0.2003 + 0.0097 + 0.0227 + 0.0764}{4} = \frac{0.3091}{4} = 0.0773$$

$$\text{denom} = \sqrt{0.0773 + 10^{-6}} = 0.2780$$

$$\text{normalized} = [\frac{0.4475}{0.2780} = 1.6098, \frac{-0.0987}{0.2780} = -0.3551, \frac{0.1506}{0.2780} = 0.5419, \frac{0.2764}{0.2780} = 0.9944]$$

Resulting normalized matrix (shape 4×4):

<bos>	1.3652	1.0334	0.6352	0.8153
I	1.7060	0.9259	-0.2442	0.4153
love	0.0304	1.5778	1.0087	0.7014
AI	1.6098	-0.3551	0.5419	0.9944

Step 13: MoE routing scores

Expert centroid matrix E (shape 4×4):

expert0	0.9000	0.1000	0.0000	0.2000
expert1	0.1000	0.8000	0.2000	-0.1000
expert2	0.3000	0.2000	0.7000	0.0000
expert3	0.0000	-0.3000	0.1000	0.9000

E^T (shape 4×4):

$$\begin{bmatrix} 0.9000 & 0.1000 & 0.3000 & 0.0000 \\ 0.1000 & 0.8000 & 0.2000 & -0.3000 \\ 0.0000 & 0.2000 & 0.7000 & 0.1000 \\ 0.2000 & -0.1000 & 0.0000 & 0.9000 \end{bmatrix}$$

Y_n (**every token as a row**) (shape 4×4):

<bos>	1.3652	1.0334	0.6352	0.8153
I	1.7060	0.9259	-0.2442	0.4153
love	0.0304	1.5778	1.0087	0.7014
AI	1.6098	-0.3551	0.5419	0.9944

Expanded arithmetic, $score = Y_n \cdot E^T$, every token:

<bos> (Y_n row = [1.3652, 1.0334, 0.6352, 0.8153])

$$score[0] = (1.3652 \times 0.9000) + (1.0334 \times 0.1000) + (0.6352 \times 0.0000) + (0.8153 \times 0.2000) = 1.4951$$

$$score[1] = (1.3652 \times 0.1000) + (1.0334 \times 0.8000) + (0.6352 \times 0.2000) + (0.8153 \times -0.1000) = 1.0087$$

$$score[2] = (1.3652 \times 0.3000) + (1.0334 \times 0.2000) + (0.6352 \times 0.7000) + (0.8153 \times 0.0000) = 1.0609$$

$$score[3] = (1.3652 \times 0.0000) + (1.0334 \times -0.3000) + (0.6352 \times 0.1000) + (0.8153 \times 0.9000) = 0.4873$$

I (Y_n row = [1.7060, 0.9259, -0.2442, 0.4153])

$$score[0] = (1.7060 \times 0.9000) + (0.9259 \times 0.1000) + (-0.2442 \times 0.0000) + (0.4153 \times 0.2000) = 1.7111$$

$$score[1] = (1.7060 \times 0.1000) + (0.9259 \times 0.8000) + (-0.2442 \times 0.2000) + (0.4153 \times -0.1000) = 0.8209$$

$$score[2] = (1.7060 \times 0.3000) + (0.9259 \times 0.2000) + (-0.2442 \times 0.7000) + (0.4153 \times 0.0000) = 0.5261$$

$$score[3] = (1.7060 \times 0.0000) + (0.9259 \times -0.3000) + (-0.2442 \times 0.1000) + (0.4153 \times 0.9000) = 0.0716$$

love (Y_n row = [0.0304, 1.5778, 1.0087, 0.7014])

$$score[0] = (0.0304 \times 0.9000) + (1.5778 \times 0.1000) + (1.0087 \times 0.0000) + (0.7014 \times 0.2000) = 0.3254$$

$$score[1] = (0.0304 \times 0.1000) + (1.5778 \times 0.8000) + (1.0087 \times 0.2000) + (0.7014 \times -0.1000) = 1.3969$$

$$score[2] = (0.0304 \times 0.3000) + (1.5778 \times 0.2000) + (1.0087 \times 0.7000) + (0.7014 \times 0.0000) = 1.0308$$

$$score[3] = (0.0304 \times 0.0000) + (1.5778 \times -0.3000) + (1.0087 \times 0.1000) + (0.7014 \times 0.9000) = 0.2588$$

AI (Y_n row = [1.6098, -0.3551, 0.5419, 0.9944])

$$\text{score}[0] = (1.6098 \times 0.9000) + (-0.3551 \times 0.1000) + (0.5419 \times 0.0000) + (0.9944 \times 0.2000) = 1.6122$$

$$\text{score}[1] = (1.6098 \times 0.1000) + (-0.3551 \times 0.8000) + (0.5419 \times 0.2000) + (0.9944 \times -0.1000) = -0.1142$$

$$\text{score}[2] = (1.6098 \times 0.3000) + (-0.3551 \times 0.2000) + (0.5419 \times 0.7000) + (0.9944 \times 0.0000) = 0.7912$$

$$\text{score}[3] = (1.6098 \times 0.0000) + (-0.3551 \times -0.3000) + (0.5419 \times 0.1000) + (0.9944 \times 0.9000) = 1.0557$$

Resulting score matrix (shape 4×4):

<bos>	1.4951	1.0087	1.0609	0.4873
I	1.7111	0.8209	0.5261	0.0716
love	0.3254	1.3969	1.0308	0.2588
AI	1.6122	-0.1142	0.7912	1.0557

Step 13b: Add routing bias (routing decision only, not the final gate value)

$$b = [0.0000, 0.0500, -0.0200, 0.1000]$$

$$\text{<bos>: scores} = [1.4951, 1.0087, 1.0609, 0.4873], \text{ biased} = [1.4951, 1.0587, 1.0409, 0.5873]$$

$$I: \text{scores} = [1.7111, 0.8209, 0.5261, 0.0716], \text{ biased} = [1.7111, 0.8709, 0.5061, 0.1716]$$

$$\text{love: scores} = [0.3254, 1.3969, 1.0308, 0.2588], \text{ biased} = [0.3254, 1.4469, 1.0108, 0.3588]$$

$$AI: \text{scores} = [1.6122, -0.1142, 0.7912, 1.0557], \text{ biased} = [1.6122, -0.0642, 0.7712, 1.1557]$$

Step 13c: Top-2 expert selection per token (by biased score)

- **<bos>**: selected experts [0, 1]
- **I**: selected experts [0, 1]
- **love**: selected experts [1, 2]
- **AI**: selected experts [0, 3]

Step 13d: Gate values (sigmoid of ORIGINAL unbiased scores, renormalized over selected experts)

<bos>:

$$\text{sigmoid}(1.4951) = 0.8168, \text{sigmoid}(1.0087) = 0.7328$$

$$\text{gates} = \left[\frac{0.8168}{1.5496} = 0.5271, \frac{0.7328}{1.5496} = 0.4729 \right]$$

I:

$$\text{sigmoid}(1.7111) = 0.8470, \text{sigmoid}(0.8209) = 0.6944$$

$$\text{gates} = \left[\frac{0.8470}{1.5414} = 0.5495, \frac{0.6944}{1.5414} = 0.4505 \right]$$

love:

$$\text{sigmoid}(1.3969) = 0.8017, \text{sigmoid}(1.0308) = 0.7371$$

$$\text{gates} = \left[\frac{0.8017}{1.5388} = 0.5210, \frac{0.7371}{1.5388} = 0.4790 \right]$$

AI:

$$\text{sigmoid}(1.6122) = 0.8337, \text{sigmoid}(1.0557) = 0.7419$$

$$\text{gates} = \left[\frac{0.8337}{1.5756} = 0.5291, \frac{0.7419}{1.5756} = 0.4709 \right]$$

Step 14: SHARED EXPERT (runs for every token)

W_1^{shared} (shape 4×4):

$$\begin{bmatrix} -0.0500 & 0.1320 & -0.3000 & -0.1190 \\ -0.2120 & -0.2450 & -0.1880 & -0.0930 \\ -0.0620 & 0.0230 & -0.0480 & 0.1110 \\ -0.1770 & 0.2270 & -0.2840 & 0.1020 \end{bmatrix}$$

Yn (every token as a row) (shape 4×4):

<bos>		1.3652	1.0334	0.6352	0.8153
I		1.7060	0.9259	-0.2442	0.4153
love		0.0304	1.5778	1.0087	0.7014
AI		1.6098	-0.3551	0.5419	0.9944

Expanded arithmetic, $hidden = Yn \cdot W_1^{shared}$, every token:

<bos> (Yn row = [1.3652, 1.0334, 0.6352, 0.8153])

$$hidden[0] = (1.3652 \times -0.0500) + (1.0334 \times -0.2120) + (0.6352 \times -0.0620) + (0.8153 \times -0.1770) = -0.4710$$

$$hidden[1] = (1.3652 \times 0.1320) + (1.0334 \times -0.2450) + (0.6352 \times 0.0230) + (0.8153 \times 0.2270) = 0.1267$$

$$hidden[2] = (1.3652 \times -0.3000) + (1.0334 \times -0.1880) + (0.6352 \times -0.0480) + (0.8153 \times -0.2840) = -0.8659$$

$$hidden[3] = (1.3652 \times -0.1190) + (1.0334 \times -0.0930) + (0.6352 \times 0.1110) + (0.8153 \times 0.1020) = -0.1049$$

I (Yn row = [1.7060, 0.9259, -0.2442, 0.4153])

$$hidden[0] = (1.7060 \times -0.0500) + (0.9259 \times -0.2120) + (-0.2442 \times -0.0620) + (0.4153 \times -0.1770) = -0.3400$$

$$hidden[1] = (1.7060 \times 0.1320) + (0.9259 \times -0.2450) + (-0.2442 \times 0.0230) + (0.4153 \times 0.2270) = 0.0870$$

$$hidden[2] = (1.7060 \times -0.3000) + (0.9259 \times -0.1880) + (-0.2442 \times -0.0480) + (0.4153 \times -0.2840) = -0.7921$$

$$hidden[3] = (1.7060 \times -0.1190) + (0.9259 \times -0.0930) + (-0.2442 \times 0.1110) + (0.4153 \times 0.1020) = -0.2739$$

love (Yn row = [0.0304, 1.5778, 1.0087, 0.7014])

$$hidden[0] = (0.0304 \times -0.0500) + (1.5778 \times -0.2120) + (1.0087 \times -0.0620) + (0.7014 \times -0.1770) = -0.5227$$

$$hidden[1] = (0.0304 \times 0.1320) + (1.5778 \times -0.2450) + (1.0087 \times 0.0230) + (0.7014 \times 0.2270) = -0.2001$$

$$hidden[2] = (0.0304 \times -0.3000) + (1.5778 \times -0.1880) + (1.0087 \times -0.0480) + (0.7014 \times -0.2840) = -0.5534$$

$$hidden[3] = (0.0304 \times -0.1190) + (1.5778 \times -0.0930) + (1.0087 \times 0.1110) + (0.7014 \times 0.1020) = 0.0332$$

AI (Yn row = [1.6098, -0.3551, 0.5419, 0.9944])

$$hidden[0] = (1.6098 \times -0.0500) + (-0.3551 \times -0.2120) + (0.5419 \times -0.0620) + (0.9944 \times -0.1770) = -0.2148$$

$$hidden[1] = (1.6098 \times 0.1320) + (-0.3551 \times -0.2450) + (0.5419 \times 0.0230) + (0.9944 \times 0.2270) = 0.5377$$

$$hidden[2] = (1.6098 \times -0.3000) + (-0.3551 \times -0.1880) + (0.5419 \times -0.0480) + (0.9944 \times -0.2840) = -0.7246$$

$$hidden[3] = (1.6098 \times -0.1190) + (-0.3551 \times -0.0930) + (0.5419 \times 0.1110) + (0.9944 \times 0.1020) = 0.0030$$

Resulting hidden matrix (shape 4×4):

$$\begin{array}{c|cccc} <bos> & -0.4710 & 0.1267 & -0.8659 & -0.1049 \\ I & -0.3400 & 0.0870 & -0.7921 & -0.2739 \\ love & -0.5227 & -0.2001 & -0.5534 & 0.0332 \\ AI & -0.2148 & 0.5377 & -0.7246 & 0.0030 \end{array}$$

Hidden matrix BEFORE GELU (shape 4×4):

$$\begin{array}{c|cccc} <bos> & -0.4710 & 0.1267 & -0.8659 & -0.1049 \\ I & -0.3400 & 0.0870 & -0.7921 & -0.2739 \\ love & -0.5227 & -0.2001 & -0.5534 & 0.0332 \\ AI & -0.2148 & 0.5377 & -0.7246 & 0.0030 \end{array}$$

$$GELU(x) = 0.5x \left(1 + \tanh \left(\sqrt{2/\pi} (x + 0.044715x^3) \right) \right)$$

<bos>:

$$[-0.4710 \rightarrow -0.1502, 0.1267 \rightarrow 0.0697, -0.8659 \rightarrow -0.1675, -0.1049 \rightarrow -0.0481]$$

I:

$$[-0.3400 \rightarrow -0.1247, 0.0870 \rightarrow 0.0465, -0.7921 \rightarrow -0.1697, -0.2739 \rightarrow -0.1074]$$

love:

$$[-0.5227 \rightarrow -0.1571, -0.2001 \rightarrow -0.0842, -0.5534 \rightarrow -0.1605, 0.0332 \rightarrow 0.0170]$$

AI:

$$[-0.2148 \rightarrow -0.0891, 0.5377 \rightarrow 0.3788, -0.7246 \rightarrow -0.1699, 0.0030 \rightarrow 0.0015]$$

Resulting activated matrix (shape 4×4):

$$\begin{array}{c|cccc} <bos> & -0.1502 & 0.0697 & -0.1675 & -0.0481 \\ I & -0.1247 & 0.0465 & -0.1697 & -0.1074 \\ love & -0.1571 & -0.0842 & -0.1605 & 0.0170 \\ AI & -0.0891 & 0.3788 & -0.1699 & 0.0015 \end{array}$$

W_2^{shared} (shape 4×4):

$$\begin{bmatrix} -0.0500 & 0.0350 & -0.2160 & -0.1810 \\ 0.1800 & 0.2810 & -0.1120 & 0.1150 \\ 0.2260 & 0.2370 & -0.2490 & -0.2770 \\ -0.1980 & 0.2270 & -0.2410 & -0.0470 \end{bmatrix}$$

activated (every token as a row) (shape 4×4):

$$\begin{array}{c|cccc} <bos> & -0.1502 & 0.0697 & -0.1675 & -0.0481 \\ I & -0.1247 & 0.0465 & -0.1697 & -0.1074 \\ love & -0.1571 & -0.0842 & -0.1605 & 0.0170 \\ AI & -0.0891 & 0.3788 & -0.1699 & 0.0015 \end{array}$$

Expanded arithmetic, $FFN^{shared} = activated \cdot W_2^{shared}$, every token:

<bos> (*activated* row = $[-0.1502, 0.0697, -0.1675, -0.0481]$)

$$FFN^{shared}[0] = (-0.1502 \times -0.0500) + (0.0697 \times 0.1800) + (-0.1675 \times 0.2260) + (-0.0481 \times -0.1980) = -0.0083$$

$$FFN^{shared}[1] = (-0.1502 \times 0.0350) + (0.0697 \times 0.2810) + (-0.1675 \times 0.2370) + (-0.0481 \times 0.2270) = -0.0363$$

$$FFN^{shared}[2] = (-0.1502 \times -0.2160) + (0.0697 \times -0.1120) + (-0.1675 \times -0.2490) + (-0.0481 \times -0.2410) = 0.0779$$

$$FFN^{shared}[3] = (-0.1502 \times -0.1810) + (0.0697 \times 0.1150) + (-0.1675 \times -0.2770) + (-0.0481 \times -0.0470) = 0.0838$$

I (*activated* row = $[-0.1247, 0.0465, -0.1697, -0.1074]$)

$$FFN^{shared}[0] = (-0.1247 \times -0.0500) + (0.0465 \times 0.1800) + (-0.1697 \times 0.2260) + (-0.1074 \times -0.1980) = -0.0025$$

$$FFN^{shared}[1] = (-0.1247 \times 0.0350) + (0.0465 \times 0.2810) + (-0.1697 \times 0.2370) + (-0.1074 \times 0.2270) = -0.0559$$

$$FFN^{shared}[2] = (-0.1247 \times -0.2160) + (0.0465 \times -0.1120) + (-0.1697 \times -0.2490) + (-0.1074 \times -0.2410) = 0.0899$$

$$FFN^{shared}[3] = (-0.1247 \times -0.1810) + (0.0465 \times 0.1150) + (-0.1697 \times -0.2770) + (-0.1074 \times -0.0470) = 0.0800$$

love (*activated* row = $[-0.1571, -0.0842, -0.1605, 0.0170]$)

$$FFN^{shared}[0] = (-0.1571 \times -0.0500) + (-0.0842 \times 0.1800) + (-0.1605 \times 0.2260) + (0.0170 \times -0.1980) = -0.0469$$

$$FFN^{shared}[1] = (-0.1571 \times 0.0350) + (-0.0842 \times 0.2810) + (-0.1605 \times 0.2370) + (0.0170 \times 0.2270) = -0.0633$$

$$FFN^{shared}[2] = (-0.1571 \times -0.2160) + (-0.0842 \times -0.1120) + (-0.1605 \times -0.2490) + (0.0170 \times -0.2410) = 0.0792$$

$$FFN^{shared}[3] = (-0.1571 \times -0.1810) + (-0.0842 \times 0.1150) + (-0.1605 \times -0.2770) + (0.0170 \times -0.0470) = 0.0624$$

AI (*activated* row = $[-0.0891, 0.3788, -0.1699, 0.0015]$)

$$FFN^{shared}[0] = (-0.0891 \times -0.0500) + (0.3788 \times 0.1800) + (-0.1699 \times 0.2260) + (0.0015 \times -0.1980) = 0.0340$$

$$FFN^{shared}[1] = (-0.0891 \times 0.0350) + (0.3788 \times 0.2810) + (-0.1699 \times 0.2370) + (0.0015 \times 0.2270) = 0.0634$$

$$FFN^{shared}[2] = (-0.0891 \times -0.2160) + (0.3788 \times -0.1120) + (-0.1699 \times -0.2490) + (0.0015 \times -0.2410) = 0.0188$$

$$FFN^{shared}[3] = (-0.0891 \times -0.1810) + (0.3788 \times 0.1150) + (-0.1699 \times -0.2770) + (0.0015 \times -0.0470) = 0.1067$$

Resulting FFN^{shared} matrix (shape 4×4):

$$\begin{array}{c|cccc} \text{<bos>} & -0.0083 & -0.0363 & 0.0779 & 0.0838 \\ \text{I} & -0.0025 & -0.0559 & 0.0899 & 0.0800 \\ \text{love} & -0.0469 & -0.0633 & 0.0792 & 0.0624 \\ \text{AI} & 0.0340 & 0.0634 & 0.0188 & 0.1067 \end{array}$$

Step 15: ROUTED EXPERTS – every (token, selected-expert) pair

Token <bos> through Expert 0

W_1^{e0} (shape 4×4):

$$\begin{bmatrix} 0.1630 & -0.2880 & 0.0800 & 0.1490 \\ -0.0010 & -0.1650 & -0.1810 & 0.1560 \\ -0.1990 & -0.2470 & 0.1110 & 0.2720 \\ -0.2980 & 0.0070 & 0.1880 & 0.0680 \end{bmatrix}$$

Yn (every token as a row) (shape 1×4):

$$\text{<bos>} \mid \begin{bmatrix} 1.3652 & 1.0334 & 0.6352 & 0.8153 \end{bmatrix}$$

Expanded arithmetic, $hidden = Yn \cdot W_1^{e0}$, every token:

<bos> (Yn row = $[1.3652, 1.0334, 0.6352, 0.8153]$)

$$hidden[0] = (1.3652 \times 0.1630) + (1.0334 \times -0.0010) + (0.6352 \times -0.1990) + (0.8153 \times -0.2980) = -0.1479$$

$$hidden[1] = (1.3652 \times -0.2880) + (1.0334 \times -0.1650) + (0.6352 \times -0.2470) + (0.8153 \times 0.0070) = -0.7149$$

$$hidden[2] = (1.3652 \times 0.0800) + (1.0334 \times -0.1810) + (0.6352 \times 0.1110) + (0.8153 \times 0.1880) = 0.1460$$

$$hidden[3] = (1.3652 \times 0.1490) + (1.0334 \times 0.1560) + (0.6352 \times 0.2720) + (0.8153 \times 0.0680) = 0.5928$$

Resulting $hidden$ matrix (shape 1×4):

$$\text{<bos>} \mid \begin{bmatrix} -0.1479 & -0.7149 & 0.1460 & 0.5928 \end{bmatrix}$$

Hidden matrix BEFORE GELU (shape 1×4):

$$\text{<bos>} \mid \begin{bmatrix} -0.1479 & -0.7149 & 0.1460 & 0.5928 \end{bmatrix}$$

$$\text{GELU}(x) = 0.5x \left(1 + \tanh \left(\sqrt{2/\pi} (x + 0.044715x^3) \right) \right)$$

<bos> :

$$[-0.1479 \rightarrow -0.0653, -0.7149 \rightarrow -0.1697, 0.1460 \rightarrow 0.0815, 0.5928 \rightarrow 0.4288]$$

Resulting activated matrix (shape 1×4):

$$\text{<bos>} \mid \begin{bmatrix} -0.0653 & -0.1697 & 0.0815 & 0.4288 \end{bmatrix}$$

W_2^{e0} (shape 4×4):

$$\begin{bmatrix} 0.1330 & -0.1250 & 0.2510 & 0.1290 \\ 0.0260 & -0.2150 & -0.0760 & 0.1040 \\ -0.0350 & -0.0400 & 0.0710 & 0.0080 \\ 0.0900 & 0.0610 & 0.1830 & 0.0130 \end{bmatrix}$$

$activated$ (every token as a row) (shape 1×4):

$$\langle \text{bos} \rangle \mid \begin{bmatrix} -0.0653 & -0.1697 & 0.0815 & 0.4288 \end{bmatrix}$$

Expanded arithmetic, $FFN = \text{activated} \cdot W_2^{e0}$, every token:

$$\langle \text{bos} \rangle (\text{activated row} = [-0.0653, -0.1697, 0.0815, 0.4288])$$

$$FFN[0] = (-0.0653 \times 0.1330) + (-0.1697 \times 0.0260) + (0.0815 \times -0.0350) + (0.4288 \times 0.0900) = 0.0227$$

$$FFN[1] = (-0.0653 \times -0.1250) + (-0.1697 \times -0.2150) + (0.0815 \times -0.0400) + (0.4288 \times 0.0610) = 0.0675$$

$$FFN[2] = (-0.0653 \times 0.2510) + (-0.1697 \times -0.0760) + (0.0815 \times 0.0710) + (0.4288 \times 0.1830) = 0.0808$$

$$FFN[3] = (-0.0653 \times 0.1290) + (-0.1697 \times 0.1040) + (0.0815 \times 0.0080) + (0.4288 \times 0.0130) = -0.0198$$

Resulting FFN matrix (shape 1×4):

$$\langle \text{bos} \rangle \mid \begin{bmatrix} 0.0227 & 0.0675 & 0.0808 & -0.0198 \end{bmatrix}$$

Token $\langle \text{bos} \rangle$ through Expert 1

$$W_1^{e1} (\text{shape } 4 \times 4):$$

$$\begin{bmatrix} -0.1920 & -0.2880 & -0.0220 & 0.1350 \\ -0.0480 & -0.0090 & -0.2920 & -0.0080 \\ 0.2650 & 0.2100 & 0.1380 & -0.2350 \\ 0.2360 & 0.2140 & -0.2010 & 0.0790 \end{bmatrix}$$

Yn (every token as a row) (shape 1×4):

$$\langle \text{bos} \rangle \mid \begin{bmatrix} 1.3652 & 1.0334 & 0.6352 & 0.8153 \end{bmatrix}$$

Expanded arithmetic, $hidden = Yn \cdot W_1^{e1}$, every token:

$$\langle \text{bos} \rangle (Yn \text{ row} = [1.3652, 1.0334, 0.6352, 0.8153])$$

$$hidden[0] = (1.3652 \times -0.1920) + (1.0334 \times -0.0480) + (0.6352 \times 0.2650) + (0.8153 \times 0.2360) = 0.0490$$

$$hidden[1] = (1.3652 \times -0.2880) + (1.0334 \times -0.0090) + (0.6352 \times 0.2100) + (0.8153 \times 0.2140) = -0.0946$$

$$hidden[2] = (1.3652 \times -0.0220) + (1.0334 \times -0.2920) + (0.6352 \times 0.1380) + (0.8153 \times -0.2010) = -0.4080$$

$$hidden[3] = (1.3652 \times 0.1350) + (1.0334 \times -0.0080) + (0.6352 \times -0.2350) + (0.8153 \times 0.0790) = 0.0912$$

Resulting $hidden$ matrix (shape 1×4):

$$\langle \text{bos} \rangle \mid \begin{bmatrix} 0.0490 & -0.0946 & -0.4080 & 0.0912 \end{bmatrix}$$

Hidden matrix BEFORE GELU (shape 1×4):

$$\langle \text{bos} \rangle \mid \begin{bmatrix} 0.0490 & -0.0946 & -0.4080 & 0.0912 \end{bmatrix}$$

$$\text{GELU}(x) = 0.5x \left(1 + \tanh \left(\sqrt{2/\pi} (x + 0.044715x^3) \right) \right)$$

$\langle \text{bos} \rangle$:

$$[0.0490 \rightarrow 0.0255, -0.0946 \rightarrow -0.0437, -0.4080 \rightarrow -0.1394, 0.0912 \rightarrow 0.0489]$$

Resulting activated matrix (shape 1×4):

$$\langle \text{bos} \rangle \mid \begin{bmatrix} 0.0255 & -0.0437 & -0.1394 & 0.0489 \end{bmatrix}$$

W_2^{e1} (shape 4×4):

$$\begin{bmatrix} -0.2880 & -0.2300 & -0.1100 & -0.2050 \\ 0.1550 & 0.1910 & -0.0930 & -0.1090 \\ -0.2330 & -0.2500 & 0.1280 & 0.0600 \\ -0.2670 & -0.0120 & -0.0590 & 0.2090 \end{bmatrix}$$

activated (every token as a row) (shape 1×4):

$$\langle \text{bos} \rangle \mid \begin{bmatrix} 0.0255 & -0.0437 & -0.1394 & 0.0489 \end{bmatrix}$$

Expanded arithmetic, $FFN = \text{activated} \cdot W_2^{e1}$, every token:

$$\langle \text{bos} \rangle (\text{activated row} = [0.0255, -0.0437, -0.1394, 0.0489])$$

$$FFN[0] = (0.0255 \times -0.2880) + (-0.0437 \times 0.1550) + (-0.1394 \times -0.2330) + (0.0489 \times -0.2670) = 0.0053$$

$$FFN[1] = (0.0255 \times -0.2300) + (-0.0437 \times 0.1910) + (-0.1394 \times -0.2500) + (0.0489 \times -0.0120) = 0.0200$$

$$FFN[2] = (0.0255 \times -0.1100) + (-0.0437 \times -0.0930) + (-0.1394 \times 0.1280) + (0.0489 \times -0.0590) = -0.0195$$

$$FFN[3] = (0.0255 \times -0.2050) + (-0.0437 \times -0.1090) + (-0.1394 \times 0.0600) + (0.0489 \times 0.2090) = 0.0014$$

Resulting FFN matrix (shape 1×4):

$$\langle \text{bos} \rangle \mid \begin{bmatrix} 0.0053 & 0.0200 & -0.0195 & 0.0014 \end{bmatrix}$$

Token I through Expert 0

W_1^{e0} (shape 4×4):

$$\begin{bmatrix} 0.1630 & -0.2880 & 0.0800 & 0.1490 \\ -0.0010 & -0.1650 & -0.1810 & 0.1560 \\ -0.1990 & -0.2470 & 0.1110 & 0.2720 \\ -0.2980 & 0.0070 & 0.1880 & 0.0680 \end{bmatrix}$$

Yn (every token as a row) (shape 1×4):

$$I \mid \begin{bmatrix} 1.7060 & 0.9259 & -0.2442 & 0.4153 \end{bmatrix}$$

Expanded arithmetic, $hidden = Yn \cdot W_1^{e0}$, every token:

$$I (Yn \text{ row} = [1.7060, 0.9259, -0.2442, 0.4153])$$

$$hidden[0] = (1.7060 \times 0.1630) + (0.9259 \times -0.0010) + (-0.2442 \times -0.1990) + (0.4153 \times -0.2980) = 0.2020$$

$$hidden[1] = (1.7060 \times -0.2880) + (0.9259 \times -0.1650) + (-0.2442 \times -0.2470) + (0.4153 \times 0.0070) = -0.5809$$

$$hidden[2] = (1.7060 \times 0.0800) + (0.9259 \times -0.1810) + (-0.2442 \times 0.1110) + (0.4153 \times 0.1880) = 0.0199$$

$$hidden[3] = (1.7060 \times 0.1490) + (0.9259 \times 0.1560) + (-0.2442 \times 0.2720) + (0.4153 \times 0.0680) = 0.3605$$

Resulting *hidden* matrix (shape 1×4):

$$I \mid \begin{array}{cccc} 0.2020 & -0.5809 & 0.0199 & 0.3605 \end{array}$$

Hidden matrix BEFORE GELU (shape 1×4):

$$I \mid \begin{array}{cccc} 0.2020 & -0.5809 & 0.0199 & 0.3605 \end{array}$$

$$\text{GELU}(x) = 0.5x \left(1 + \tanh \left(\sqrt{2/\pi} (x + 0.044715x^3) \right) \right)$$

I:

$$[0.2020 \rightarrow 0.1172, -0.5809 \rightarrow -0.1631, 0.0199 \rightarrow 0.0101, 0.3605 \rightarrow 0.2310]$$

Resulting activated matrix (shape 1×4):

$$I \mid \begin{array}{cccc} 0.1172 & -0.1631 & 0.0101 & 0.2310 \end{array}$$

W_2^{e0} (shape 4×4):

$$\begin{bmatrix} 0.1330 & -0.1250 & 0.2510 & 0.1290 \\ 0.0260 & -0.2150 & -0.0760 & 0.1040 \\ -0.0350 & -0.0400 & 0.0710 & 0.0080 \\ 0.0900 & 0.0610 & 0.1830 & 0.0130 \end{bmatrix}$$

activated (**every token as a row**) (shape 1×4):

$$I \mid \begin{array}{cccc} 0.1172 & -0.1631 & 0.0101 & 0.2310 \end{array}$$

Expanded arithmetic, $FFN = activated \cdot W_2^{e0}$, every token:

I (*activated* row = $[0.1172, -0.1631, 0.0101, 0.2310]$)

$$FFN[0] = (0.1172 \times 0.1330) + (-0.1631 \times 0.0260) + (0.0101 \times -0.0350) + (0.2310 \times 0.0900) = 0.0318$$

$$FFN[1] = (0.1172 \times -0.1250) + (-0.1631 \times -0.2150) + (0.0101 \times -0.0400) + (0.2310 \times 0.0610) = 0.0341$$

$$FFN[2] = (0.1172 \times 0.2510) + (-0.1631 \times -0.0760) + (0.0101 \times 0.0710) + (0.2310 \times 0.1830) = 0.0848$$

$$FFN[3] = (0.1172 \times 0.1290) + (-0.1631 \times 0.1040) + (0.0101 \times 0.0080) + (0.2310 \times 0.0130) = 0.0012$$

Resulting *FFN* matrix (shape 1×4):

$$I \mid \begin{array}{cccc} 0.0318 & 0.0341 & 0.0848 & 0.0012 \end{array}$$

Token I through Expert 1

W_I^{e1} (shape 4×4):

$$\begin{bmatrix} -0.1920 & -0.2880 & -0.0220 & 0.1350 \\ -0.0480 & -0.0090 & -0.2920 & -0.0080 \\ 0.2650 & 0.2100 & 0.1380 & -0.2350 \\ 0.2360 & 0.2140 & -0.2010 & 0.0790 \end{bmatrix}$$

Yn (**every token as a row**) (shape 1×4):

$$I \mid \begin{bmatrix} 1.7060 & 0.9259 & -0.2442 & 0.4153 \end{bmatrix}$$

Expanded arithmetic, $hidden = Yn \cdot W_1^{e1}$, every token:

I (Yn row = $[1.7060, 0.9259, -0.2442, 0.4153]$)

$$hidden[0] = (1.7060 \times -0.1920) + (0.9259 \times -0.0480) + (-0.2442 \times 0.2650) + (0.4153 \times 0.2360) = -0.3387$$

$$hidden[1] = (1.7060 \times -0.2880) + (0.9259 \times -0.0090) + (-0.2442 \times 0.2100) + (0.4153 \times 0.2140) = -0.4621$$

$$hidden[2] = (1.7060 \times -0.0220) + (0.9259 \times -0.2920) + (-0.2442 \times 0.1380) + (0.4153 \times -0.2010) = -0.4251$$

$$hidden[3] = (1.7060 \times 0.1350) + (0.9259 \times -0.0080) + (-0.2442 \times -0.2350) + (0.4153 \times 0.0790) = 0.3131$$

Resulting $hidden$ matrix (shape 1×4):

$$I \mid \begin{bmatrix} -0.3387 & -0.4621 & -0.4251 & 0.3131 \end{bmatrix}$$

Hidden matrix BEFORE GELU (shape 1×4):

$$I \mid \begin{bmatrix} -0.3387 & -0.4621 & -0.4251 & 0.3131 \end{bmatrix}$$

$$GELU(x) = 0.5x \left(1 + \tanh \left(\sqrt{2/\pi} (x + 0.044715x^3) \right) \right)$$

I :

$$[-0.3387 \rightarrow -0.1244, -0.4621 \rightarrow -0.1488, -0.4251 \rightarrow -0.1426, 0.3131 \rightarrow 0.1950]$$

Resulting activated matrix (shape 1×4):

$$I \mid \begin{bmatrix} -0.1244 & -0.1488 & -0.1426 & 0.1950 \end{bmatrix}$$

W_2^{e1} (shape 4×4):

$$\begin{bmatrix} -0.2880 & -0.2300 & -0.1100 & -0.2050 \\ 0.1550 & 0.1910 & -0.0930 & -0.1090 \\ -0.2330 & -0.2500 & 0.1280 & 0.0600 \\ -0.2670 & -0.0120 & -0.0590 & 0.2090 \end{bmatrix}$$

activated (every token as a row) (shape 1×4):

$$I \mid \begin{bmatrix} -0.1244 & -0.1488 & -0.1426 & 0.1950 \end{bmatrix}$$

Expanded arithmetic, $FFN = activated \cdot W_2^{e1}$, every token:

I (*activated row* = $[-0.1244, -0.1488, -0.1426, 0.1950]$)

$$FFN[0] = (-0.1244 \times -0.2880) + (-0.1488 \times 0.1550) + (-0.1426 \times -0.2330) + (0.1950 \times -0.2670) = -0.0061$$

$$FFN[1] = (-0.1244 \times -0.2300) + (-0.1488 \times 0.1910) + (-0.1426 \times -0.2500) + (0.1950 \times -0.0120) = 0.0335$$

$$FFN[2] = (-0.1244 \times -0.1100) + (-0.1488 \times -0.0930) + (-0.1426 \times 0.1280) + (0.1950 \times -0.0590) = -0.0022$$

$$FFN[3] = (-0.1244 \times -0.2050) + (-0.1488 \times -0.1090) + (-0.1426 \times 0.0600) + (0.1950 \times 0.2090) = 0.0739$$

Resulting FFN matrix (shape 1×4):

$$I \mid \begin{bmatrix} -0.0061 & 0.0335 & -0.0022 & 0.0739 \end{bmatrix}$$

Token love through Expert 1

W_1^{e1} (shape 4×4):

$$\begin{bmatrix} -0.1920 & -0.2880 & -0.0220 & 0.1350 \\ -0.0480 & -0.0090 & -0.2920 & -0.0080 \\ 0.2650 & 0.2100 & 0.1380 & -0.2350 \\ 0.2360 & 0.2140 & -0.2010 & 0.0790 \end{bmatrix}$$

Yn (**every token as a row**) (shape 1×4):

$$\text{love} \mid \begin{bmatrix} 0.0304 & 1.5778 & 1.0087 & 0.7014 \end{bmatrix}$$

Expanded arithmetic, $hidden = Yn \cdot W_1^{e1}$, every token:

$love$ (Yn row = $[0.0304, 1.5778, 1.0087, 0.7014]$)

$$hidden[0] = (0.0304 \times -0.1920) + (1.5778 \times -0.0480) + (1.0087 \times 0.2650) + (0.7014 \times 0.2360) = 0.3513$$

$$hidden[1] = (0.0304 \times -0.2880) + (1.5778 \times -0.0090) + (1.0087 \times 0.2100) + (0.7014 \times 0.2140) = 0.3390$$

$$hidden[2] = (0.0304 \times -0.0220) + (1.5778 \times -0.2920) + (1.0087 \times 0.1380) + (0.7014 \times -0.2010) = -0.4632$$

$$hidden[3] = (0.0304 \times 0.1350) + (1.5778 \times -0.0080) + (1.0087 \times -0.2350) + (0.7014 \times 0.0790) = -0.1902$$

Resulting $hidden$ matrix (shape 1×4):

$$\text{love} \mid \begin{bmatrix} 0.3513 & 0.3390 & -0.4632 & -0.1902 \end{bmatrix}$$

Hidden matrix BEFORE GELU (shape 1×4):

$$\text{love} \mid \begin{bmatrix} 0.3513 & 0.3390 & -0.4632 & -0.1902 \end{bmatrix}$$

$$\text{GELU}(x) = 0.5x \left(1 + \tanh \left(\sqrt{2/\pi} (x + 0.044715x^3) \right) \right)$$

$love$:

$$[0.3513 \rightarrow 0.2239, 0.3390 \rightarrow 0.2145, -0.4632 \rightarrow -0.1490, -0.1902 \rightarrow -0.0807]$$

Resulting activated matrix (shape 1×4):

$$\text{love} \mid \begin{bmatrix} 0.2239 & 0.2145 & -0.1490 & -0.0807 \end{bmatrix}$$

W_2^{e1} (shape 4×4):

$$\begin{bmatrix} -0.2880 & -0.2300 & -0.1100 & -0.2050 \\ 0.1550 & 0.1910 & -0.0930 & -0.1090 \\ -0.2330 & -0.2500 & 0.1280 & 0.0600 \\ -0.2670 & -0.0120 & -0.0590 & 0.2090 \end{bmatrix}$$

activated (**every token as a row**) (shape 1×4):

$$\text{love} \mid \begin{bmatrix} 0.2239 & 0.2145 & -0.1490 & -0.0807 \end{bmatrix}$$

Expanded arithmetic, $FFN = \text{activated} \cdot W_2^{e1}$, every token:

love (*activated* row = $[0.2239, 0.2145, -0.1490, -0.0807]$)

$$FFN[0] = (0.2239 \times -0.2880) + (0.2145 \times 0.1550) + (-0.1490 \times -0.2330) + (-0.0807 \times -0.2670) = 0.0250$$

$$FFN[1] = (0.2239 \times -0.2300) + (0.2145 \times 0.1910) + (-0.1490 \times -0.2500) + (-0.0807 \times -0.0120) = 0.0277$$

$$FFN[2] = (0.2239 \times -0.1100) + (0.2145 \times -0.0930) + (-0.1490 \times 0.1280) + (-0.0807 \times -0.0590) = -0.0589$$

$$FFN[3] = (0.2239 \times -0.2050) + (0.2145 \times -0.1090) + (-0.1490 \times 0.0600) + (-0.0807 \times 0.2090) = -0.0951$$

Resulting FFN matrix (shape 1×4):

$$\text{love} \mid \begin{bmatrix} 0.0250 & 0.0277 & -0.0589 & -0.0951 \end{bmatrix}$$

Token love through Expert 2

W_1^{e2} (shape 4×4):

$$\begin{bmatrix} -0.2080 & 0.1440 & -0.1420 & 0.0200 \\ -0.2910 & 0.2510 & 0.2400 & -0.2800 \\ 0.2740 & -0.2180 & -0.1300 & 0.0640 \\ 0.2670 & 0.2120 & -0.2990 & 0.0130 \end{bmatrix}$$

Yn (**every token as a row**) (shape 1×4):

$$\text{love} \mid \begin{bmatrix} 0.0304 & 1.5778 & 1.0087 & 0.7014 \end{bmatrix}$$

Expanded arithmetic, $hidden = Yn \cdot W_1^{e2}$, every token:

love (*Yn* row = $[0.0304, 1.5778, 1.0087, 0.7014]$)

$$hidden[0] = (0.0304 \times -0.2080) + (1.5778 \times -0.2910) + (1.0087 \times 0.2740) + (0.7014 \times 0.2670) = -0.0018$$

$$hidden[1] = (0.0304 \times 0.1440) + (1.5778 \times 0.2510) + (1.0087 \times -0.2180) + (0.7014 \times 0.2120) = 0.3292$$

$$hidden[2] = (0.0304 \times -0.1420) + (1.5778 \times 0.2400) + (1.0087 \times -0.1300) + (0.7014 \times -0.2990) = 0.0335$$

$$hidden[3] = (0.0304 \times 0.0200) + (1.5778 \times -0.2800) + (1.0087 \times 0.0640) + (0.7014 \times 0.0130) = -0.3675$$

Resulting *hidden* matrix (shape 1×4):

$$\text{love} \mid \begin{bmatrix} -0.0018 & 0.3292 & 0.0335 & -0.3675 \end{bmatrix}$$

Hidden matrix BEFORE GELU (shape 1×4):

$$\text{love} \mid \begin{bmatrix} -0.0018 & 0.3292 & 0.0335 & -0.3675 \end{bmatrix}$$

$$\text{GELU}(x) = 0.5x \left(1 + \tanh \left(\sqrt{2/\pi} (x + 0.044715x^3) \right) \right)$$

love:

$$[-0.0018 \rightarrow -0.0009, 0.3292 \rightarrow 0.2071, 0.0335 \rightarrow 0.0172, -0.3675 \rightarrow -0.1311]$$

Resulting activated matrix (shape 1×4):

$$\text{love} \mid \begin{bmatrix} -0.0009 & 0.2071 & 0.0172 & -0.1311 \end{bmatrix}$$

W_2^{e2} (shape 4×4):

$$\begin{bmatrix} 0.0310 & -0.0090 & 0.1610 & -0.2040 \\ 0.1590 & -0.2880 & -0.2190 & -0.2300 \\ -0.1140 & 0.1030 & -0.0170 & 0.1900 \\ -0.1260 & 0.1400 & 0.1220 & -0.1030 \end{bmatrix}$$

activated (**every token as a row**) (shape 1×4):

$$\text{love} \mid \begin{bmatrix} -0.0009 & 0.2071 & 0.0172 & -0.1311 \end{bmatrix}$$

Expanded arithmetic, $FFN = activated \cdot W_2^{e2}$, every token:

love (*activated* row = $[-0.0009, 0.2071, 0.0172, -0.1311]$)

$$FFN[0] = (-0.0009 \times 0.0310) + (0.2071 \times 0.1590) + (0.0172 \times -0.1140) + (-0.1311 \times -0.1260) = 0.0474$$

$$FFN[1] = (-0.0009 \times -0.0090) + (0.2071 \times -0.2880) + (0.0172 \times 0.1030) + (-0.1311 \times 0.1400) = -0.0762$$

$$FFN[2] = (-0.0009 \times 0.1610) + (0.2071 \times -0.2190) + (0.0172 \times -0.0170) + (-0.1311 \times 0.1220) = -0.0618$$

$$FFN[3] = (-0.0009 \times -0.2040) + (0.2071 \times -0.2300) + (0.0172 \times 0.1900) + (-0.1311 \times -0.1030) = -0.0307$$

Resulting FFN matrix (shape 1×4):

$$\text{love} \mid \begin{bmatrix} 0.0474 & -0.0762 & -0.0618 & -0.0307 \end{bmatrix}$$

Token AI through Expert 0

W_1^{e0} (shape 4×4):

$$\begin{bmatrix} 0.1630 & -0.2880 & 0.0800 & 0.1490 \\ -0.0010 & -0.1650 & -0.1810 & 0.1560 \\ -0.1990 & -0.2470 & 0.1110 & 0.2720 \\ -0.2980 & 0.0070 & 0.1880 & 0.0680 \end{bmatrix}$$

Yn (**every token as a row**) (shape 1×4):

$$AI \mid \begin{bmatrix} 1.6098 & -0.3551 & 0.5419 & 0.9944 \end{bmatrix}$$

Expanded arithmetic, $hidden = Yn \cdot W_1^{e0}$, every token:

AI (Yn row = $[1.6098, -0.3551, 0.5419, 0.9944]$)

$$hidden[0] = (1.6098 \times 0.1630) + (-0.3551 \times -0.0010) + (0.5419 \times -0.1990) + (0.9944 \times -0.2980) = -0.1414$$

$$hidden[1] = (1.6098 \times -0.2880) + (-0.3551 \times -0.1650) + (0.5419 \times -0.2470) + (0.9944 \times 0.0070) = -0.5319$$

$$hidden[2] = (1.6098 \times 0.0800) + (-0.3551 \times -0.1810) + (0.5419 \times 0.1110) + (0.9944 \times 0.1880) = 0.4401$$

$$hidden[3] = (1.6098 \times 0.1490) + (-0.3551 \times 0.1560) + (0.5419 \times 0.2720) + (0.9944 \times 0.0680) = 0.3995$$

Resulting $hidden$ matrix (shape 1×4):

$$AI \mid \begin{bmatrix} -0.1414 & -0.5319 & 0.4401 & 0.3995 \end{bmatrix}$$

Hidden matrix BEFORE GELU (shape 1×4):

$$AI \mid \begin{bmatrix} -0.1414 & -0.5319 & 0.4401 & 0.3995 \end{bmatrix}$$

$$GELU(x) = 0.5x \left(1 + \tanh \left(\sqrt{2/\pi} (x + 0.044715x^3) \right) \right)$$

AI :

$$[-0.1414 \rightarrow -0.0628, -0.5319 \rightarrow -0.1582, 0.4401 \rightarrow 0.2949, 0.3995 \rightarrow 0.2617]$$

Resulting activated matrix (shape 1×4):

$$AI \mid \begin{bmatrix} -0.0628 & -0.1582 & 0.2949 & 0.2617 \end{bmatrix}$$

W_2^{e0} (shape 4×4):

$$\begin{bmatrix} 0.1330 & -0.1250 & 0.2510 & 0.1290 \\ 0.0260 & -0.2150 & -0.0760 & 0.1040 \\ -0.0350 & -0.0400 & 0.0710 & 0.0080 \\ 0.0900 & 0.0610 & 0.1830 & 0.0130 \end{bmatrix}$$

$activated$ (**every token as a row**) (shape 1×4):

$$AI \mid \begin{bmatrix} -0.0628 & -0.1582 & 0.2949 & 0.2617 \end{bmatrix}$$

Expanded arithmetic, $FFN = activated \cdot W_2^{e0}$, every token:

AI (*activated row* = $[-0.0628, -0.1582, 0.2949, 0.2617]$)

$$FFN[0] = (-0.0628 \times 0.1330) + (-0.1582 \times 0.0260) + (0.2949 \times -0.0350) + (0.2617 \times 0.0900) = 0.0008$$

$$FFN[1] = (-0.0628 \times -0.1250) + (-0.1582 \times -0.2150) + (0.2949 \times -0.0400) + (0.2617 \times 0.0610) = 0.0460$$

$$FFN[2] = (-0.0628 \times 0.2510) + (-0.1582 \times -0.0760) + (0.2949 \times 0.0710) + (0.2617 \times 0.1830) = 0.0651$$

$$FFN[3] = (-0.0628 \times 0.1290) + (-0.1582 \times 0.1040) + (0.2949 \times 0.0080) + (0.2617 \times 0.0130) = -0.0188$$

Resulting FFN matrix (shape 1×4):

$$AI \mid \begin{bmatrix} 0.0008 & 0.0460 & 0.0651 & -0.0188 \end{bmatrix}$$

Token AI through Expert 3

W_1^{e3} (shape 4×4):

$$\begin{bmatrix} 0.1670 & -0.1570 & 0.1950 & 0.2790 \\ 0.2840 & -0.0280 & 0.0650 & 0.1650 \\ 0.0850 & 0.1330 & -0.2790 & -0.1210 \\ -0.2650 & 0.2140 & -0.0760 & 0.1080 \end{bmatrix}$$

Yn (**every token as a row**) (shape 1×4):

$$AI \mid \begin{bmatrix} 1.6098 & -0.3551 & 0.5419 & 0.9944 \end{bmatrix}$$

Expanded arithmetic, $hidden = Yn \cdot W_1^{e3}$, every token:

AI (Yn row = $[1.6098, -0.3551, 0.5419, 0.9944]$)

$$hidden[0] = (1.6098 \times 0.1670) + (-0.3551 \times 0.2840) + (0.5419 \times 0.0850) + (0.9944 \times -0.2650) = -0.0495$$

$$hidden[1] = (1.6098 \times -0.1570) + (-0.3551 \times -0.0280) + (0.5419 \times 0.1330) + (0.9944 \times 0.2140) = 0.0421$$

$$hidden[2] = (1.6098 \times 0.1950) + (-0.3551 \times 0.0650) + (0.5419 \times -0.2790) + (0.9944 \times -0.0760) = 0.0641$$

$$hidden[3] = (1.6098 \times 0.2790) + (-0.3551 \times 0.1650) + (0.5419 \times -0.1210) + (0.9944 \times 0.1080) = 0.4324$$

Resulting $hidden$ matrix (shape 1×4):

$$AI \mid \begin{bmatrix} -0.0495 & 0.0421 & 0.0641 & 0.4324 \end{bmatrix}$$

Hidden matrix BEFORE GELU (shape 1×4):

$$AI \mid \begin{bmatrix} -0.0495 & 0.0421 & 0.0641 & 0.4324 \end{bmatrix}$$

$$GELU(x) = 0.5x \left(1 + \tanh \left(\sqrt{2/\pi} (x + 0.044715x^3) \right) \right)$$

AI :

$$[-0.0495 \rightarrow -0.0238, 0.0421 \rightarrow 0.0217, 0.0641 \rightarrow 0.0337, 0.4324 \rightarrow 0.2885]$$

Resulting activated matrix (shape 1×4):

$$\text{AI} \mid \begin{bmatrix} -0.0238 & 0.0217 & 0.0337 & 0.2885 \end{bmatrix}$$

W_2^{e3} (shape 4×4):

$$\begin{bmatrix} -0.1460 & -0.0910 & -0.2940 & -0.0850 \\ 0.2690 & -0.1690 & -0.1080 & 0.2510 \\ -0.2810 & -0.2610 & 0.0780 & 0.2240 \\ -0.2950 & 0.1480 & 0.1880 & -0.2550 \end{bmatrix}$$

activated (**every token as a row**) (shape 1×4):

$$\text{AI} \mid \begin{bmatrix} -0.0238 & 0.0217 & 0.0337 & 0.2885 \end{bmatrix}$$

Expanded arithmetic, $FFN = \text{activated} \cdot W_2^{e3}$, **every token**:

AI (*activated* row = $[-0.0238, 0.0217, 0.0337, 0.2885]$)

$$FFN[0] = (-0.0238 \times -0.1460) + (0.0217 \times 0.2690) + (0.0337 \times -0.2810) + (0.2885 \times -0.2950) = -0.0853$$

$$FFN[1] = (-0.0238 \times -0.0910) + (0.0217 \times -0.1690) + (0.0337 \times -0.2610) + (0.2885 \times 0.1480) = 0.0324$$

$$FFN[2] = (-0.0238 \times -0.2940) + (0.0217 \times -0.1080) + (0.0337 \times 0.0780) + (0.2885 \times 0.1880) = 0.0615$$

$$FFN[3] = (-0.0238 \times -0.0850) + (0.0217 \times 0.2510) + (0.0337 \times 0.2240) + (0.2885 \times -0.2550) = -0.0585$$

Resulting FFN matrix (shape 1×4):

$$\text{AI} \mid \begin{bmatrix} -0.0853 & 0.0324 & 0.0615 & -0.0585 \end{bmatrix}$$

Step 16: Combine routed experts by gate, then full MoE output

<bos>:

$$\text{expert 0} : 0.5271 \times [0.0227, 0.0675, 0.0808, -0.0198] = [0.0119, 0.0356, 0.0426, -0.0105]$$

$$\text{expert 1} : 0.4729 \times [0.0053, 0.0200, -0.0195, 0.0014] = [0.0025, 0.0095, -0.0092, 0.0007]$$

$$\text{routed sum} = [0.0144, 0.0451, 0.0334, -0.0098]$$

I:

$$\text{expert 0} : 0.5495 \times [0.0318, 0.0341, 0.0848, 0.0012] = [0.0175, 0.0187, 0.0466, 0.0007]$$

$$\text{expert 1} : 0.4505 \times [-0.0061, 0.0335, -0.0022, 0.0739] = [-0.0027, 0.0151, -0.0010, 0.0333]$$

$$\text{routed sum} = [0.0147, 0.0338, 0.0456, 0.0340]$$

love:

$$\text{expert 1} : 0.5210 \times [0.0250, 0.0277, -0.0589, -0.0951] = [0.0130, 0.0144, -0.0307, -0.0495]$$

$$\text{expert 2} : 0.4790 \times [0.0474, -0.0762, -0.0618, -0.0307] = [0.0227, -0.0365, -0.0296, -0.0147]$$

$$\text{routed sum} = [0.0358, -0.0221, -0.0603, -0.0642]$$

AI:

$$\begin{aligned} \text{expert 0} &: 0.5291 \times [0.0008, 0.0460, 0.0651, -0.0188] = [0.0004, 0.0244, 0.0345, -0.0099] \\ \text{expert 3} &: 0.4709 \times [-0.0853, 0.0324, 0.0615, -0.0585] = [-0.0401, 0.0153, 0.0290, -0.0276] \\ \text{routed sum} &= [-0.0397, 0.0396, 0.0634, -0.0375] \end{aligned}$$

Step 16b: Full MoE output = Y (residual) + shared_expert(Yn) + routed_sum

Y (residual stream) (shape 4×4):

<bos>	0.2880	0.2180	0.1340	0.1720
I	0.3619	0.1964	-0.0518	0.0881
love	0.0069	0.3589	0.2294	0.1595
AI	0.4475	-0.0987	0.1506	0.2764

shared_expert(Yn) (shape 4×4):

<bos>	-0.0083	-0.0363	0.0779	0.0838
I	-0.0025	-0.0559	0.0899	0.0800
love	-0.0469	-0.0633	0.0792	0.0624
AI	0.0340	0.0634	0.0188	0.1067

routed_sum (shape 4×4):

<bos>	0.0144	0.0451	0.0334	-0.0098
I	0.0147	0.0338	0.0456	0.0340
love	0.0358	-0.0221	-0.0603	-0.0642
AI	-0.0397	0.0396	0.0634	-0.0375

Resulting full MoE output matrix (shape 4×4):

<bos>	0.2941	0.2268	0.2453	0.2460
I	0.3741	0.1743	0.0837	0.2021
love	-0.0043	0.2735	0.2484	0.1577
AI	0.4417	0.0043	0.2328	0.3456

Step 17: Final RMSNorm -> LAYER OUTPUT

Input matrix (every token as a row) (shape 4×4):

<bos>	0.2941	0.2268	0.2453	0.2460
I	0.3741	0.1743	0.0837	0.2021
love	-0.0043	0.2735	0.2484	0.1577
AI	0.4417	0.0043	0.2328	0.3456

$$\langle \text{bos} \rangle: v = [0.2941, 0.2268, 0.2453, 0.2460]$$

$$\text{mean}(v^2) = \frac{0.0865 + 0.0514 + 0.0602 + 0.0605}{4} = \frac{0.2586}{4} = 0.0647$$

$$\begin{aligned}
& \text{denom} = \sqrt{0.0647 + 10^{-6}} = 0.2543 \\
& \text{normalized} = \left[\frac{0.2941}{0.2543} = 1.1567, \frac{0.2268}{0.2543} = 0.8919, \frac{0.2453}{0.2543} = 0.9645, \frac{0.2460}{0.2543} = 0.9675 \right] \\
& \text{I: } v = [0.3741, 0.1743, 0.0837, 0.2021] \\
& \text{mean}(v^2) = \frac{0.1400 + 0.0304 + 0.0070 + 0.0408}{4} = \frac{0.2182}{4} = 0.0546 \\
& \text{denom} = \sqrt{0.0546 + 10^{-6}} = 0.2336 \\
& \text{normalized} = \left[\frac{0.3741}{0.2336} = 1.6019, \frac{0.1743}{0.2336} = 0.7464, \frac{0.0837}{0.2336} = 0.3582, \frac{0.2021}{0.2336} = 0.8652 \right] \\
& \text{love: } v = [-0.0043, 0.2735, 0.2484, 0.1577] \\
& \text{mean}(v^2) = \frac{0.0000 + 0.0748 + 0.0617 + 0.0249}{4} = \frac{0.1614}{4} = 0.0403 \\
& \text{denom} = \sqrt{0.0403 + 10^{-6}} = 0.2009 \\
& \text{normalized} = \left[\frac{-0.0043}{0.2009} = -0.0212, \frac{0.2735}{0.2009} = 1.3614, \frac{0.2484}{0.2009} = 1.2367, \frac{0.1577}{0.2009} = 0.7852 \right] \\
& \text{AI: } v = [0.4417, 0.0043, 0.2328, 0.3456] \\
& \text{mean}(v^2) = \frac{0.1951 + 0.0000 + 0.0542 + 0.1194}{4} = \frac{0.3688}{4} = 0.0922 \\
& \text{denom} = \sqrt{0.0922 + 10^{-6}} = 0.3036 \\
& \text{normalized} = \left[\frac{0.4417}{0.3036} = 1.4548, \frac{0.0043}{0.3036} = 0.0142, \frac{0.2328}{0.3036} = 0.7667, \frac{0.3456}{0.3036} = 1.1382 \right]
\end{aligned}$$

Resulting normalized matrix (shape 4×4):

<bos>	1.1567	0.8919	0.9645	0.9675
I	1.6019	0.7464	0.3582	0.8652
love	-0.0212	1.3614	1.2367	0.7852
AI	1.4548	0.0142	0.7667	1.1382

Step 18: Main-model next-token prediction, every position

Every position’s final hidden state predicts the token that should come right after it. This is the single most important step in the whole network: it’s the only place a hidden vector gets compared against actual candidate words and turned into a probability.

The core idea: comparing two vectors that live in the same space. The same embedding table is used at both ends of the network. When a word like “AI” is first looked up, it becomes a vector living in the model’s embedding space. If, after all the attention and MoE processing, a position’s final hidden state also lives in that same space, it can be directly compared against every candidate word’s embedding – and “compared” just means a dot product. Every word is an arrow pointing in some direction; the hidden state is also an arrow in that same space, and the word whose arrow points most closely in the same direction is the model’s best guess for what comes next. This mechanism only becomes genuinely predictive through training – with these hand-picked toy weights, the specific numbers below are not meaningful, only the *mechanism* connecting a hidden vector to a predicted word is real.

Emb (tied embedding/output matrix) (shape 4×4):

a	0.1500	0.0500	-0.0500	0.1000
lot	0.0500	-0.1500	0.2000	0.0500
AI	0.3000	-0.2000	0.1000	0.2000
love	-0.1000	0.3000	0.2000	0.1000

Emb^T (shape 4×4):

$$\begin{bmatrix} 0.1500 & 0.0500 & 0.3000 & -0.1000 \\ 0.0500 & -0.1500 & -0.2000 & 0.3000 \\ -0.0500 & 0.2000 & 0.1000 & 0.2000 \\ 0.1000 & 0.0500 & 0.2000 & 0.1000 \end{bmatrix}$$

$final_output$ (every token as a row) (shape 4×4):

<bos>	1.1567	0.8919	0.9645	0.9675
I	1.6019	0.7464	0.3582	0.8652
love	-0.0212	1.3614	1.2367	0.7852
AI	1.4548	0.0142	0.7667	1.1382

Expanded arithmetic, $logit = final_output \cdot Emb^T$, every token:

<bos> ($final_output$ row = [1.1567, 0.8919, 0.9645, 0.9675])

$$logit[0] = (1.1567 \times 0.1500) + (0.8919 \times 0.0500) + (0.9645 \times -0.0500) + (0.9675 \times 0.1000) = 0.2666$$

$$logit[1] = (1.1567 \times 0.0500) + (0.8919 \times -0.1500) + (0.9645 \times 0.2000) + (0.9675 \times 0.0500) = 0.1653$$

$$logit[2] = (1.1567 \times 0.3000) + (0.8919 \times -0.2000) + (0.9645 \times 0.1000) + (0.9675 \times 0.2000) = 0.4586$$

$$logit[3] = (1.1567 \times -0.1000) + (0.8919 \times 0.3000) + (0.9645 \times 0.2000) + (0.9675 \times 0.1000) = 0.4416$$

I ($final_output$ row = [1.6019, 0.7464, 0.3582, 0.8652])

$$logit[0] = (1.6019 \times 0.1500) + (0.7464 \times 0.0500) + (0.3582 \times -0.0500) + (0.8652 \times 0.1000) = 0.3462$$

$$logit[1] = (1.6019 \times 0.0500) + (0.7464 \times -0.1500) + (0.3582 \times 0.2000) + (0.8652 \times 0.0500) = 0.0830$$

$$logit[2] = (1.6019 \times 0.3000) + (0.7464 \times -0.2000) + (0.3582 \times 0.1000) + (0.8652 \times 0.2000) = 0.5401$$

$$logit[3] = (1.6019 \times -0.1000) + (0.7464 \times 0.3000) + (0.3582 \times 0.2000) + (0.8652 \times 0.1000) = 0.2219$$

love ($final_output$ row = [-0.0212, 1.3614, 1.2367, 0.7852])

$$logit[0] = (-0.0212 \times 0.1500) + (1.3614 \times 0.0500) + (1.2367 \times -0.0500) + (0.7852 \times 0.1000) = 0.0816$$

$$logit[1] = (-0.0212 \times 0.0500) + (1.3614 \times -0.1500) + (1.2367 \times 0.2000) + (0.7852 \times 0.0500) = 0.0813$$

$$logit[2] = (-0.0212 \times 0.3000) + (1.3614 \times -0.2000) + (1.2367 \times 0.1000) + (0.7852 \times 0.2000) = 0.0021$$

$$logit[3] = (-0.0212 \times -0.1000) + (1.3614 \times 0.3000) + (1.2367 \times 0.2000) + (0.7852 \times 0.1000) = 0.7364$$

AI ($final_output$ row = [1.4548, 0.0142, 0.7667, 1.1382])

$$\text{logit}[0] = (1.4548 \times 0.1500) + (0.0142 \times 0.0500) + (0.7667 \times -0.0500) + (1.1382 \times 0.1000) = 0.2944$$

$$\text{logit}[1] = (1.4548 \times 0.0500) + (0.0142 \times -0.1500) + (0.7667 \times 0.2000) + (1.1382 \times 0.0500) = 0.2809$$

$$\text{logit}[2] = (1.4548 \times 0.3000) + (0.0142 \times -0.2000) + (0.7667 \times 0.1000) + (1.1382 \times 0.2000) = 0.7379$$

$$\text{logit}[3] = (1.4548 \times -0.1000) + (0.0142 \times 0.3000) + (0.7667 \times 0.2000) + (1.1382 \times 0.1000) = 0.1259$$

Resulting logit matrix (shape 4×4):

	<i>a</i>	<i>lot</i>	<i>AI</i>	<i>love</i>
<bos>	0.2666	0.1653	0.4586	0.4416
I	0.3462	0.0830	0.5401	0.2219
love	0.0816	0.0813	0.0021	0.7364
AI	0.2944	0.2809	0.7379	0.1259

Step 18b: Softmax over the vocabulary, every position, fully explained

Five steps turn a row of logits into a genuine probability distribution: (1) find the max, (2) subtract it from every logit — a numerical-stability trick that changes nothing mathematically, since subtracting a constant before exponentiating multiplies numerator and denominator by the same factor, which cancels, (3) exponentiate every shifted value, (4) sum the exponentials, (5) divide each exponential by that sum.

Position <bos>

$$\text{logits} = [\text{a} = 0.2666, \text{lot} = 0.1653, \text{AI} = 0.4586, \text{love} = 0.4416]$$

$$\text{max} = 0.4586 \quad \text{shifted} = [-0.1920, -0.2933, 0.0000, -0.0170]$$

$$\text{exponentiate} = [e^{-0.1920} = 0.8253, e^{-0.2933} = 0.7458, e^{0.0000} = 1.0000, e^{-0.0170} = 0.9831] \quad \text{sum} = 3.5543$$

$$\text{probabilities} = [\frac{0.8253}{3.5543} = 0.2322, \frac{0.7458}{3.5543} = 0.2098, \frac{1.0000}{3.5543} = 0.2814, \frac{0.9831}{3.5543} = 0.2766]$$

predicted next token after <bos>: AI (probability 0.2814)

Position I

$$\text{logits} = [\text{a} = 0.3462, \text{lot} = 0.0830, \text{AI} = 0.5401, \text{love} = 0.2219]$$

$$\text{max} = 0.5401 \quad \text{shifted} = [-0.1939, -0.4571, 0.0000, -0.3182]$$

$$\text{exponentiate} = [e^{-0.1939} = 0.8237, e^{-0.4571} = 0.6331, e^{0.0000} = 1.0000, e^{-0.3182} = 0.7274] \quad \text{sum} = 3.1843$$

$$\text{probabilities} = [\frac{0.8237}{3.1843} = 0.2587, \frac{0.6331}{3.1843} = 0.1988, \frac{1.0000}{3.1843} = 0.3140, \frac{0.7274}{3.1843} = 0.2284]$$

predicted next token after I: AI (probability 0.3140)

Position love

$$\text{logits} = [\text{a} = 0.0816, \text{lot} = 0.0813, \text{AI} = 0.0021, \text{love} = 0.7364]$$

$$\text{max} = 0.7364 \quad \text{shifted} = [-0.6548, -0.6551, -0.7344, 0.0000]$$

$$\text{exponentiate} = [e^{-0.6548} = 0.5195, e^{-0.6551} = 0.5194, e^{-0.7344} = 0.4798, e^{0.0000} = 1.0000] \quad \text{sum} = 2.5187$$

$$\text{probabilities} = [\frac{0.5195}{2.5187} = 0.2063, \frac{0.5194}{2.5187} = 0.2062, \frac{0.4798}{2.5187} = 0.1905, \frac{1.0000}{2.5187} = 0.3970]$$

predicted next token after love: love (probability 0.3970)

Position AI

$\text{logits} = [\text{a} = 0.2944, \text{lot} = 0.2809, \text{AI} = 0.7379, \text{love} = 0.1259]$
 $\text{max} = 0.7379 \quad \text{shifted} = [-0.4435, -0.4570, 0.0000, -0.6120]$
 $\text{exponentiate} = [e^{-0.4435} = 0.6418, e^{-0.4570} = 0.6331, e^{0.0000} = 1.0000, e^{-0.6120} = 0.5423] \quad \text{sum} = 2.8172$
 $\text{probabilities} = [\frac{0.6418}{2.8172} = 0.2278, \frac{0.6331}{2.8172} = 0.2247, \frac{1.0000}{2.8172} = 0.3550, \frac{0.5423}{2.8172} = 0.1925]$
predicted next token after AI: AI (probability 0.3550)

Step 19: MTP module (depth 1) – predict t6 using true t5 embedding

$\text{RMSNorm}(h_0 \text{ at AI}) = [1.4548, 0.0142, 0.7667, 1.1382]$
 $\text{RMSNorm}(\text{true } t_5 \text{ embedding 'a'}) = [1.5491, 0.5164, -0.5164, 1.0327]$

Concatenated [RMSNorm(h0) ; RMSNorm(Emb(t5))] (shape 1×8):

concat_AI | 1.4548 0.0142 0.7667 1.1382 1.5491 0.5164 -0.5164 1.0327

Project through M1

M_1 (shape 8×4):

$$\begin{bmatrix} -0.2540 & 0.1680 & -0.0370 & 0.1340 \\ 0.2870 & 0.0230 & 0.0010 & -0.2570 \\ -0.1390 & -0.0000 & 0.1080 & 0.1820 \\ -0.0710 & -0.2600 & -0.1270 & 0.2460 \\ -0.1720 & -0.0290 & 0.2590 & -0.2850 \\ 0.0600 & 0.2700 & -0.1620 & 0.0290 \\ 0.2450 & -0.2200 & 0.0140 & 0.1500 \\ 0.1010 & -0.0190 & -0.1770 & -0.0060 \end{bmatrix}$$

concat (**every token as a row**) (shape 1×8):

concat_AI | 1.4548 0.0142 0.7667 1.1382 1.5491 0.5164 -0.5164 1.0327

Expanded arithmetic, $h' = \text{concat} \cdot M_1$, every token:

concat_AI (*concat* row = [1.4548, 0.0142, 0.7667, 1.1382, 1.5491, 0.5164, -0.5164, 1.0327])

$$h'[0] = (1.4548 \times -0.2540) + (0.0142 \times 0.2870) + (0.7667 \times -0.1390) + (1.1382 \times -0.0710) + (1.5491 \times -0.1720) + (0.5164 \times 0.0600) + (-0.5164 \times 0.2450) + (1.0327 \times 0.1010) = -0.8105$$

$$h'[1] = (1.4548 \times 0.1680) + (0.0142 \times 0.0230) + (0.7667 \times -0.0000) + (1.1382 \times -0.2600) + (1.5491 \times -0.0290) + (0.5164 \times 0.2700) + (-0.5164 \times -0.2200) + (1.0327 \times -0.0190) = 0.1373$$

$$h'[2] = (1.4548 \times -0.0370) + (0.0142 \times 0.0010) + (0.7667 \times 0.1080) + (1.1382 \times -0.1270) + (1.5491 \times 0.2590) + (0.5164 \times -0.1620) + (-0.5164 \times 0.0140) + (1.0327 \times -0.1770) = 0.0120$$

$$h'[3] = (1.4548 \times 0.1340) + (0.0142 \times -0.2570) + (0.7667 \times 0.1820) + (1.1382 \times 0.2460) + (1.5491 \times -0.2850) + (0.5164 \times 0.0290) + (-0.5164 \times 0.1500) + (1.0327 \times -0.0060) = 0.1007$$

Resulting h' matrix (shape 1×4):

$$\text{concat_AI} \mid \begin{bmatrix} -0.8105 & 0.1373 & 0.0120 & 0.1007 \end{bmatrix}$$

MTP's tiny transformer block: first linear layer

W_1^{mtp} (shape 4×4):

$$\begin{bmatrix} 0.2240 & 0.2810 & 0.2220 & 0.0190 \\ -0.1600 & -0.2930 & -0.0420 & -0.0590 \\ 0.0140 & -0.0130 & 0.0330 & 0.0260 \\ 0.1570 & 0.1270 & 0.0720 & -0.0440 \end{bmatrix}$$

h' (**every token as a row**) (shape 1×4):

$$h' \mid \begin{bmatrix} -0.8105 & 0.1373 & 0.0120 & 0.1007 \end{bmatrix}$$

Expanded arithmetic, $hidden = h' \cdot W_1^{mtp}$, every token:

h' (h' row = $[-0.8105, 0.1373, 0.0120, 0.1007]$)

$$hidden[0] = (-0.8105 \times 0.2240) + (0.1373 \times -0.1600) + (0.0120 \times 0.0140) + (0.1007 \times 0.1570) = -0.1875$$

$$hidden[1] = (-0.8105 \times 0.2810) + (0.1373 \times -0.2930) + (0.0120 \times -0.0130) + (0.1007 \times 0.1270) = -0.2553$$

$$hidden[2] = (-0.8105 \times 0.2220) + (0.1373 \times -0.0420) + (0.0120 \times 0.0330) + (0.1007 \times 0.0720) = -0.1781$$

$$hidden[3] = (-0.8105 \times 0.0190) + (0.1373 \times -0.0590) + (0.0120 \times 0.0260) + (0.1007 \times -0.0440) = -0.0276$$

Resulting $hidden$ matrix (shape 1×4):

$$h' \mid \begin{bmatrix} -0.1875 & -0.2553 & -0.1781 & -0.0276 \end{bmatrix}$$

GELU activation

Hidden matrix BEFORE GELU (shape 1×4):

$$h' \mid \begin{bmatrix} -0.1875 & -0.2553 & -0.1781 & -0.0276 \end{bmatrix}$$

$$\text{GELU}(x) = 0.5x \left(1 + \tanh \left(\sqrt{2/\pi} (x + 0.044715x^3) \right) \right)$$

h' :

$$[-0.1875 \rightarrow -0.0798, -0.2553 \rightarrow -0.1019, -0.1781 \rightarrow -0.0764, -0.0276 \rightarrow -0.0135]$$

Resulting activated matrix (shape 1×4):

$$h' \mid \begin{bmatrix} -0.0798 & -0.1019 & -0.0764 & -0.0135 \end{bmatrix}$$

Second linear layer

W_2^{mtp} (shape 4×4):

$$\begin{bmatrix} -0.2940 & 0.0010 & -0.0030 & -0.2200 \\ -0.2150 & -0.1690 & -0.0490 & -0.1510 \\ -0.2500 & -0.0930 & -0.2000 & 0.2270 \\ 0.2710 & -0.2770 & 0.1190 & 0.0440 \end{bmatrix}$$

activated (every token as a row) (shape 1×4):

$$h' \mid \begin{bmatrix} -0.0798 & -0.1019 & -0.0764 & -0.0135 \end{bmatrix}$$

Expanded arithmetic, $FFN = activated \cdot W_2^{mtp}$, every token:

h' (*activated* row = $[-0.0798, -0.1019, -0.0764, -0.0135]$)

$$FFN[0] = (-0.0798 \times -0.2940) + (-0.1019 \times -0.2150) + (-0.0764 \times -0.2500) + (-0.0135 \times 0.2710) = 0.0608$$

$$FFN[1] = (-0.0798 \times 0.0010) + (-0.1019 \times -0.1690) + (-0.0764 \times -0.0930) + (-0.0135 \times -0.2770) = 0.0280$$

$$FFN[2] = (-0.0798 \times -0.0030) + (-0.1019 \times -0.0490) + (-0.0764 \times -0.2000) + (-0.0135 \times 0.1190) = 0.0189$$

$$FFN[3] = (-0.0798 \times -0.2200) + (-0.1019 \times -0.1510) + (-0.0764 \times 0.2270) + (-0.0135 \times 0.0440) = 0.0150$$

Resulting FFN matrix (shape 1×4):

$$h' \mid \begin{bmatrix} 0.0608 & 0.0280 & 0.0189 & 0.0150 \end{bmatrix}$$

$$\text{residual} = [-0.8105, 0.1373, 0.0120, 0.1007] + [0.0608, 0.0280, 0.0189, 0.0150] = [-0.7497, 0.1653, 0.0309, 0.1157]$$

$$h_1^{final} = [-1.9298, 0.4254, 0.0795, 0.2977]$$

MTP logits and prediction for t6

Emb^T (shape 4×4):

$$\begin{bmatrix} 0.1500 & 0.0500 & 0.3000 & -0.1000 \\ 0.0500 & -0.1500 & -0.2000 & 0.3000 \\ -0.0500 & 0.2000 & 0.1000 & 0.2000 \\ 0.1000 & 0.0500 & 0.2000 & 0.1000 \end{bmatrix}$$

$h1_{final}$ (every token as a row) (shape 1×4):

$$h1_final \mid \begin{bmatrix} -1.9298 & 0.4254 & 0.0795 & 0.2977 \end{bmatrix}$$

Expanded arithmetic, $logit = h1_{final} \cdot Emb^T$, every token:

$h1_{final}$ ($h1_{final}$ row = $[-1.9298, 0.4254, 0.0795, 0.2977]$)

$$logit[0] = (-1.9298 \times 0.1500) + (0.4254 \times 0.0500) + (0.0795 \times -0.0500) + (0.2977 \times 0.1000) = -0.2424$$

$$logit[1] = (-1.9298 \times 0.0500) + (0.4254 \times -0.1500) + (0.0795 \times 0.2000) + (0.2977 \times 0.0500) = -0.1295$$

$$logit[2] = (-1.9298 \times 0.3000) + (0.4254 \times -0.2000) + (0.0795 \times 0.1000) + (0.2977 \times 0.2000) = -0.5965$$

$$\text{logit}[3] = (-1.9298 \times -0.1000) + (0.4254 \times 0.3000) + (0.0795 \times 0.2000) + (0.2977 \times 0.1000) = 0.3663$$

Resulting *logit* matrix (shape 1×4):

	<i>a</i>	<i>lot</i>	<i>AI</i>	<i>love</i>
h1_final	-0.2424	-0.1295	-0.5965	0.3663

$$\text{probabilities} = [\text{a} = 0.2146, \text{lot} = 0.2403, \text{AI} = 0.1506, \text{love} = 0.3945]$$

predicted t6 = love

Step 20: FP8 block quantization of *Wo*, every entry

***Wo*, original full-precision weight matrix** (shape 2×4):

$$\begin{bmatrix} 0.5000 & 0.2000 & 0.1000 & 0.3000 \\ 0.3000 & 0.4000 & 0.2000 & 0.1000 \end{bmatrix}$$

Treating the whole 2×4 *Wo* matrix as one block (real blocks are 128×128).

$$\text{amax} = \max(|W_o|) = 0.5000 \quad \text{scale} = \frac{448}{0.5000} = 896.0000$$

***Wo* x scale (every entry mapped into the FP8 representable range)** (shape 2×4):

$$\begin{bmatrix} 448.0000 & 179.2000 & 89.6000 & 268.8000 \\ 268.8000 & 358.4000 & 179.2000 & 89.6000 \end{bmatrix}$$

Quantizing each scaled entry to FP8 E4M3 (3 mantissa bits $\rightarrow$ 8 discrete levels per exponent range):

$$W_o \times \text{scale}[0, 0] = 448.0000 : \text{exponent} = 8, \text{ mantissa} = 1.7500 \rightarrow 1.7500 \times 2^8 = 448.0000$$

$$W_o \times \text{scale}[0, 1] = 179.2000 : \text{exponent} = 7, \text{ mantissa} = 1.4000 \rightarrow 1.3750 \times 2^7 = 176.0000$$

$$W_o \times \text{scale}[0, 2] = 89.6000 : \text{exponent} = 6, \text{ mantissa} = 1.4000 \rightarrow 1.3750 \times 2^6 = 88.0000$$

$$W_o \times \text{scale}[0, 3] = 268.8000 : \text{exponent} = 8, \text{ mantissa} = 1.0500 \rightarrow 1.0000 \times 2^8 = 256.0000$$

$$W_o \times \text{scale}[1, 0] = 268.8000 : \text{exponent} = 8, \text{ mantissa} = 1.0500 \rightarrow 1.0000 \times 2^8 = 256.0000$$

$$W_o \times \text{scale}[1, 1] = 358.4000 : \text{exponent} = 8, \text{ mantissa} = 1.4000 \rightarrow 1.3750 \times 2^8 = 352.0000$$

$$W_o \times \text{scale}[1, 2] = 179.2000 : \text{exponent} = 7, \text{ mantissa} = 1.4000 \rightarrow 1.3750 \times 2^7 = 176.0000$$

$$W_o \times \text{scale}[1, 3] = 89.6000 : \text{exponent} = 6, \text{ mantissa} = 1.4000 \rightarrow 1.3750 \times 2^6 = 88.0000$$

Quantized (still in scaled FP8 range) (shape 2×4):

$$\begin{bmatrix} 448.0000 & 176.0000 & 88.0000 & 256.0000 \\ 256.0000 & 352.0000 & 176.0000 & 88.0000 \end{bmatrix}$$

Dequantize (divide back by scale= 896.0000):

$$W_{o,quant}[0, 0] = \frac{448.0000}{896.0000} = 0.5000 \quad (\text{orig } 0.5000, \text{ err } 0.0000)$$

$$\begin{aligned}
W_{o,quant}[0,1] &= \frac{176.0000}{896.0000} = 0.1964 \quad (\text{orig } 0.2000, \text{ err } 0.0036) \\
W_{o,quant}[0,2] &= \frac{88.0000}{896.0000} = 0.0982 \quad (\text{orig } 0.1000, \text{ err } 0.0018) \\
W_{o,quant}[0,3] &= \frac{256.0000}{896.0000} = 0.2857 \quad (\text{orig } 0.3000, \text{ err } 0.0143) \\
W_{o,quant}[1,0] &= \frac{256.0000}{896.0000} = 0.2857 \quad (\text{orig } 0.3000, \text{ err } 0.0143) \\
W_{o,quant}[1,1] &= \frac{352.0000}{896.0000} = 0.3929 \quad (\text{orig } 0.4000, \text{ err } 0.0071) \\
W_{o,quant}[1,2] &= \frac{176.0000}{896.0000} = 0.1964 \quad (\text{orig } 0.2000, \text{ err } 0.0036) \\
W_{o,quant}[1,3] &= \frac{88.0000}{896.0000} = 0.0982 \quad (\text{orig } 0.1000, \text{ err } 0.0018)
\end{aligned}$$

Wo_quant, final dequantized matrix actually used (shape 2×4):

$$\begin{bmatrix} 0.5000 & 0.1964 & 0.0982 & 0.2857 \\ 0.2857 & 0.3929 & 0.1964 & 0.0982 \end{bmatrix}$$

Step 20b: Re-run the Step 10 matmul with the quantized Wo concat (attention output, same input as Step 10) (shape 4×2):

$$\begin{array}{c|cc}
<\text{bos}> & 0.3199 & 0.2599 \\
\text{I} & 0.2560 & 0.1130 \\
\text{love} & 0.1793 & 0.0576 \\
\text{AI} & 0.2045 & 0.1510
\end{array}$$

MultiHead(X) with FULL-precision Wo (shape 4×4):

$$\begin{array}{c|cccc}
<\text{bos}> & 0.2379 & 0.1679 & 0.0840 & 0.1220 \\
\text{I} & 0.1619 & 0.0964 & 0.0482 & 0.0881 \\
\text{love} & 0.1069 & 0.0589 & 0.0294 & 0.0595 \\
\text{AI} & 0.1475 & 0.1013 & 0.0507 & 0.0764
\end{array}$$

MultiHead(X) with FP8-QUANTIZED Wo (shape 4×4):

$$\begin{array}{c|cccc}
<\text{bos}> & 0.2342 & 0.1649 & 0.0825 & 0.1169 \\
\text{I} & 0.1603 & 0.0947 & 0.0473 & 0.0842 \\
\text{love} & 0.1061 & 0.0578 & 0.0289 & 0.0569 \\
\text{AI} & 0.1454 & 0.0995 & 0.0497 & 0.0733
\end{array}$$

Absolute difference (the real ‘cost’ of FP8) (shape 4×4):

$$\begin{array}{c|cccc}
<\text{bos}> & 0.0037 & 0.0030 & 0.0015 & 0.0050 \\
\text{I} & 0.0016 & 0.0017 & 0.0009 & 0.0039 \\
\text{love} & 0.0008 & 0.0011 & 0.0005 & 0.0027 \\
\text{AI} & 0.0022 & 0.0018 & 0.0009 & 0.0032
\end{array}$$

style decoder layer walkthrough in code -- tokenization, RMSNorm, Multi-head Latent Attention (with decoupled RoPE), causal multi-head attention, Mixture-of-Experts routing and expert computation, Multi-Token Prediction, and FP8 block quantization -- with hands-on exercises at the end for extending the example.

DeepSeek-Style Decoder Layer — Full Worked Example: "I love AI"

Replicates every calculation from the walkthrough in executable code: RMSNorm, Multi-Head Latent Attention (down/up-projection + decoupled RoPE), causal multi-head attention, Mixture-of-Experts (routing + auxiliary-loss-free bias + shared/routed experts), Multi-Token Prediction, and FP8 block quantization.

In [1]:

```
import numpy as np
np.set_printoptions(precision=4, suppress=True)

tokens = ["<bos>", "I", "love", "AI"]
n = len(tokens)
d_model, num_heads = 4, 2
d_c, d_cq = 2, 2
nope_dim, rope_dim = 1, 2
print(f"Tokens: {tokens} (causal decoder -- each token sees only itself and earlier tokens)")
```

Tokens: ['<bos>', 'I', 'love', 'AI'] (causal decoder -- each token sees only itself and earlier tokens)

Tokenization and embedding lookup

A real BPE tokenizer maps text to subword IDs, then looks up one row of a learned [vocab_size, d_model] embedding table per token. Here we simulate that lookup directly with toy 4-dim rows.

In [2]:

```
token_emb = {
    "<bos>": np.array([0.05,0.05,0.05,0.05]),
    "I": np.array([0.20,0.10,-0.10,0.00]),
    "love": np.array([-0.10,0.30,0.20,0.10]),
    "AI": np.array([0.30,-0.20,0.10,0.20]),
    "a": np.array([0.15,0.05,-0.05,0.10]), # ground-truth t5, used later for MTP
}
X = np.array([token_emb[t] for t in tokens])
for t,v in zip(tokens, X): print(f"{t:6s} -> {v}")
```

```
<bos> -> [0.05 0.05 0.05 0.05]
I      -> [ 0.2  0.1 -0.1  0. ]
love   -> [-0.1  0.3  0.2  0.1]
AI      -> [ 0.3 -0.2  0.1  0.2]
```

Step 3: RMSNorm (pre-norm)

$\text{RMSNorm}(x) = x / \sqrt{\text{mean}(x^2) + \text{eps}}$ — no mean-centering, unlike LayerNorm.

In [3]:

```
def rmsnorm(v, eps=1e-6):
    return v/np.sqrt(np.mean(v**2, axis=-1, keepdims=True)+eps)

Xn = rmsnorm(X)
for t,v in zip(tokens, Xn): print(f"{t:6s} -> {v}")
```

```
<bos> -> [0.9998 0.9998 0.9998 0.9998]
I      -> [ 1.6329  0.8165 -0.8165  0. ]
love   -> [-0.5164  1.5492  1.0328  0.5164]
AI      -> [ 1.4142 -0.9428  0.4714  0.9428]
```

Steps 4-5: MLA — down-project into latents, up-project into per-head content

In [4]:

```
W_DKV = np.array([[0.3,0.1],[0.2,-0.2],[0.1,0.3],[0.0,0.2]])
W_DQ = np.array([[0.2,0.0],[0.1,0.3],[0.0,0.2],[0.3,0.1]])
c_KV = Xn @ W_DKV # THIS is what would be cached at inference time
c_Q = Xn @ W_DQ
print("c_KV (cached latent):\n", c_KV)
print("c_Q (latent query):\n", c_Q)

W_UK = [np.array([[0.6],[0.3]]), np.array([[0.2],[0.7]])]
W_UQ = [np.array([[0.5],[0.4]]), np.array([[0.3],[0.6]])]
W_UV = [np.array([[0.4],[0.2]]), np.array([[0.1],[0.5]])]

heads_Kc, heads_Qc, heads_V = [], [], []
for h in range(num_heads):
    heads_Kc.append(c_KV @ W_UK[h])
    heads_Qc.append(c_Q @ W_UQ[h])
    heads_V.append(c_KV @ W_UV[h])
    print(f"Head {h+1}: K_content={heads_Kc[h].flatten()}, Q_content={heads_Qc[h].flatten()}, V={heads_V[h].flatten()}")

c_KV (cached latent):
[[ 0.5999  0.3999]
 [ 0.5715 -0.2449]
 [ 0.2582  0.0516]
 [ 0.2828  0.66 ]]
c_Q (latent query):
[[ 0.5999  0.5999]
 [ 0.4082  0.0816]
 [ 0.2066  0.7229]
 [ 0.4714 -0.0943]]
Head 1: K_content=[0.4799 0.2694 0.1704 0.3677], Q_content=[0.5399 0.2368 0.3925 0.198
], V=[0.3199 0.1796 0.1136 0.2451]
Head 2: K_content=[ 0.3999 -0.0572  0.0878  0.5185], Q_content=[0.5399 0.1715 0.4957 0
.0849], V=[ 0.2599 -0.0653  0.0516  0.3583]
```

Steps 6-7: Decoupled RoPE part — project, then rotate by position

In [5]:

```
W_QR = np.array([[0.4,0.1],[0.2,0.3],[0.1,0.2],[0.0,0.4]])
W_KR = np.array([[0.3,0.2],[0.1,0.4],[0.2,0.1],[0.3,0.0]])
Q_rope_raw = Xn @ W_QR
K_rope_raw = Xn @ W_KR

def rotate_all(raw):
    result = np.zeros_like(raw)
    for pos in range(raw.shape[0]):
        angle = pos*1.0
        c,s = np.cos(angle), np.sin(angle)
        x1,x2 = raw[pos]
        result[pos,0] = c*x1 - s*x2
        result[pos,1] = s*x1 + c*x2
    return result

Q_rope_rot = rotate_all(Q_rope_raw)
K_rope_rot = rotate_all(K_rope_raw)
print("Q_rope rotated:\n", Q_rope_rot)
print("K_rope rotated:\n", K_rope_rot)
```

```
Q_rope rotated:
[[ 0.6999  0.9998]
 [ 0.1909  0.7507]
 [-0.8372 -0.156 ]
 [-0.4666 -0.2668]]
K_rope rotated:
[[ 0.8998  0.6999]
 [-0.2604  0.6523]
 [-0.7139  0.0708]
 [-0.6934  0.1465]]
```

Step 8: Assemble full per-head Q, K = [content ; rotated rope]

In [6]:

```
heads_Q, heads_K = [], []
for h in range(num_heads):
    heads_Q.append(np.concatenate([heads_Qc[h], Q_rope_rot], axis=1))
    heads_K.append(np.concatenate([heads_Kc[h], K_rope_rot], axis=1))
    print(f"Head {h+1} full Q:\n", heads_Q[h])
    print(f"Head {h+1} full K:\n", heads_K[h])
```

Head 1 full Q:

```
[[ 0.5399  0.6999  0.9998]
 [ 0.2368  0.1909  0.7507]
 [ 0.3925 -0.8372 -0.156 ]
 [ 0.198  -0.4666 -0.2668]]
```

Head 1 full K:

```
[[ 0.4799  0.8998  0.6999]
 [ 0.2694 -0.2604  0.6523]
 [ 0.1704 -0.7139  0.0708]
 [ 0.3677 -0.6934  0.1465]]
```

Head 2 full Q:

```
[[ 0.5399  0.6999  0.9998]
 [ 0.1715  0.1909  0.7507]
 [ 0.4957 -0.8372 -0.156 ]
 [ 0.0849 -0.4666 -0.2668]]
```

Head 2 full K:

```
[[ 0.3999  0.8998  0.6999]
 [-0.0572 -0.2604  0.6523]
 [ 0.0878 -0.7139  0.0708]
 [ 0.5185 -0.6934  0.1465]]
```

Step 9: Causal scaled dot-product attention

A causal mask blocks any key at a later position than the query.

In [7]:

```
d_k = 3
def softmax(x, axis=-1):
    e = np.exp(x - np.max(x, axis=axis, keepdims=True))
    return e/e.sum(axis=axis, keepdims=True)

causal_mask = np.triu(np.ones((n,n)), k=1).astype(bool)
print("Causal mask (True = blocked):\n", causal_mask)

head_outputs = []
for h in range(num_heads):
    Q,K,V = heads_Q[h], heads_K[h], heads_V[h]
    scores = (Q @ K.T)/np.sqrt(d_k)
    scores_masked = np.where(causal_mask, -1e9, scores)
    weights = softmax(scores_masked, axis=-1)
    out = weights @ V
    head_outputs.append(out)
    print(f"--- Head {h+1} ---")
    print("softmax weights (staircase pattern from causal masking):\n", weights)
    print("head output:\n", out)
```

```
Causal mask (True = blocked):
[[False True  True  True]
 [False False True  True]
 [False False False True]
 [False False False False]]
--- Head 1 ---
softmax weights (staircase pattern from causal masking):
[[1.      0.      0.      0.    ]
 [0.5442 0.4558 0.      0.    ]
 [0.207  0.3473 0.4456 0.    ]
 [0.1774 0.2384 0.2913 0.2929]]
head output:
[[0.3199]
 [0.256 ]
 [0.1793]
 [0.2045]]
--- Head 2 ---
softmax weights (staircase pattern from causal masking):
[[1.      0.      0.      0.    ]
 [0.5483 0.4517 0.      0.    ]
 [0.2148 0.3316 0.4536 0.    ]
 [0.1753 0.236  0.2938 0.2949]]
head output:
[[0.2599]
 [0.113 ]
 [0.0576]
 [0.151  ]]
```

Steps 10-12: Concat heads, project through W_o , residual, second RMSNorm

In [8]:

```
concat = np.concatenate([head_outputs[0], head_outputs[1]], axis=1)
Wo = np.array([[0.5,0.2,0.1,0.3],[0.3,0.4,0.2,0.1]])
attn_out = concat @ Wo
Y = X + attn_out          # residual adds back to ORIGINAL X (pre-norm architecture)
Yn = rmsnorm(Y)
print("MultiHead(X):\n", attn_out)
print("Y = X + MultiHead(X):\n", Y)
print("Yn = RMSNorm(Y):\n", Yn)
```

```
MultiHead(X):
[[0.238  0.168  0.084  0.122 ]
 [0.1619 0.0964 0.0482 0.0881]
 [0.1069 0.0589 0.0294 0.0595]
 [0.1475 0.1013 0.0506 0.0764]]
Y = X + MultiHead(X):
[[ 0.288   0.218   0.134   0.172 ]
 [ 0.3619  0.1964 -0.0518  0.0881]
 [ 0.0069  0.3589  0.2294  0.1595]
 [ 0.4475 -0.0987  0.1506  0.2764]]
Yn = RMSNorm(Y):
[[ 1.3652  1.0334  0.6352  0.8153]
 [ 1.706   0.9259 -0.2442  0.4153]
 [ 0.0304  1.5778  1.0087  0.7014]
 [ 1.6098 -0.3551  0.5419  0.9944]]
```

Step 13: MoE routing — score against experts, auxiliary-loss-free bias, top-k, gating

In [9]:

```
E = np.array([
    [0.9,0.1,0.0,0.2],[0.1,0.8,0.2,-0.1],[0.3,0.2,0.7,0.0],[0.0,-0.3,0.1,0.9]
])
bias = np.array([0.0,0.05,-0.02,0.10])
scores = Yn @ E.T
biased = scores + bias
topk = 2
selected = np.argsort(-biased, axis=1)[: , :topk]
print("Raw affinity scores:\n", scores)
print("Biased scores (routing decision only):\n", biased)
print("Selected experts per token:\n", selected)
```

```
gates = np.zeros((n, topk))
for i in range(n):
    sel_scores = scores[i, selected[i]]
    sig = 1/(1+np.exp(-sel_scores))
    gates[i] = sig/sig.sum()
print("Gate values (normalized sigmoid of UNBIASED scores):\n", gates)
```

```
Raw affinity scores:
[[ 1.4951  1.0087  1.0609  0.4873]
 [ 1.7111  0.8209  0.5261  0.0716]
 [ 0.3254  1.3969  1.0308  0.2588]
 [ 1.6122 -0.1142  0.7912  1.0557]]
Biased scores (routing decision only):
[[ 1.4951  1.0587  1.0409  0.5873]
 [ 1.7111  0.8709  0.5061  0.1716]
 [ 0.3254  1.4469  1.0108  0.3588]
 [ 1.6122 -0.0642  0.7712  1.1557]]
Selected experts per token:
[[0 1]
 [0 1]
 [1 2]
 [0 3]]
Gate values (normalized sigmoid of UNBIASED scores):
[[0.5271 0.4729]
 [0.5495 0.4505]
 [0.521  0.479 ]
 [0.5291 0.4709]]
```

Steps 14-16: Shared expert (always on) + each token's routed experts, combine

In [10]:

```
d_ff = 4
def gelu(x): return 0.5*x*(1+np.tanh(np.sqrt(2/np.pi)*(x+0.044715*x**3)))
def make_ffn(seed):
    rng = np.random.RandomState(seed)
    W1 = np.round(rng.uniform(-0.3,0.3,size=(d_model,d_ff)),3)
    W2 = np.round(rng.uniform(-0.3,0.3,size=(d_ff,d_model)),3)
    return W1, W2
def run_ffn(x, W1, W2): return gelu(x @ W1) @ W2

shared_W1, shared_W2 = make_ffn(1)
expert_ffns = [make_ffn(10+i) for i in range(4)]

shared_out = run_ffn(Yn, shared_W1, shared_W2)
print("Shared expert output (every token):\n", shared_out)

moe_out = np.zeros((n, d_model))
for i,t in enumerate(tokens):
    total = np.zeros(d_model)
    for j, eid in enumerate(selected[i]):
        W1,W2 = expert_ffns[eid]
        out = run_ffn(Yn[i:i+1], W1, W2)[0]
        total += gates[i,j]*out
    moe_out[i] = total
    print(f"{t}: routed sum = {total}")

final_moe = Y + shared_out + moe_out
final_output = rmsnorm(final_moe)
print("\nFinal layer output (after final RMSNorm):\n", final_output)
```

```
Shared expert output (every token):
[[-0.0083 -0.0363  0.0779  0.0838]
 [-0.0025 -0.0559  0.0899  0.08  ]
 [-0.0469 -0.0633  0.0792  0.0624]
 [ 0.034   0.0634  0.0188  0.1067]]
<bos>: routed sum = [ 0.0144  0.0451  0.0334 -0.0098]
I: routed sum = [0.0147 0.0338 0.0456 0.034 ]
love: routed sum = [ 0.0358 -0.0221 -0.0603 -0.0642]
AI: routed sum = [-0.0397  0.0396  0.0634 -0.0375]
```

```
Final layer output (after final RMSNorm):
[[ 1.1567  0.8919  0.9645  0.9675]
 [ 1.6019  0.7464  0.3582  0.8652]
 [-0.0212  1.3614  1.2367  0.7852]
 [ 1.4548  0.0142  0.7667  1.1382]]
```

Step 18: Main-model next-token prediction, every position

In [11]:

```
vocab = {"a": np.array([0.15,0.05,-0.05,0.10]), "lot": np.array([0.05,-0.15,0.20,0.05]),
        "AI": np.array([0.30,-0.20,0.10,0.20]), "love": np.array([-0.10,0.30,0.20,0.10])}
vocab_list = list(vocab.keys())
Emb = np.array([vocab[w] for w in vocab_list])

logits_all = final_output @ Emb.T
for i,t in enumerate(tokens):
    probs = np.exp(logits_all[i]-logits_all[i].max())
    probs /= probs.sum()
    pred = vocab_list[np.argmax(probs)]
    print(f"After '{t}': probs={dict(zip(vocab_list, np.round(probs,4)))} -> predicted '{pred}'")
```

After '<bos>': probs={'a': np.float64(0.2322), 'lot': np.float64(0.2098), 'AI': np.float64(0.2814), 'love': np.float64(0.2766)} -> predicted 'AI'
After 'I': probs={'a': np.float64(0.2587), 'lot': np.float64(0.1988), 'AI': np.float64(0.314), 'love': np.float64(0.2284)} -> predicted 'AI'
After 'love': probs={'a': np.float64(0.2063), 'lot': np.float64(0.2062), 'AI': np.float64(0.1905), 'love': np.float64(0.397)} -> predicted 'love'
After 'AI': probs={'a': np.float64(0.2278), 'lot': np.float64(0.2247), 'AI': np.float64(0.355), 'love': np.float64(0.1925)} -> predicted 'AI'

Step 19: MTP module (depth 1) — predict two tokens ahead using the TRUE next-token embedding

In [12]:

```
h_AI = final_output[-1]
true_t5_emb = vocab["a"] # ground truth t5, used via teacher forcing

concat_in = np.concatenate([rmsnorm(h_AI), rmsnorm(true_t5_emb)])
M1 = np.round(np.random.RandomState(7).uniform(-0.3,0.3,size=(8,4)),3)
h_prime = concat_in @ M1

W1_mtp = np.round(np.random.RandomState(8).uniform(-0.3,0.3,size=(4,4)),3)
W2_mtp = np.round(np.random.RandomState(9).uniform(-0.3,0.3,size=(4,4)),3)
h1_ffn = gelu(h_prime @ W1_mtp) @ W2_mtp
h1_final = rmsnorm(h_prime + h1_ffn)

logits_t6 = h1_final @ Emb.T
probs_t6 = np.exp(logits_t6-logits_t6.max()); probs_t6 /= probs_t6.sum()
print("t6 probabilities:", dict(zip(vocab_list, np.round(probs_t6,4))))
print("Predicted t6:", vocab_list[np.argmax(probs_t6)])
```

t6 probabilities: {'a': np.float64(0.2146), 'lot': np.float64(0.2403), 'AI': np.float64(0.1506), 'love': np.float64(0.3945)}
Predicted t6: love

Step 20: FP8 block quantization applied to W_o , then re-run the real Step-10 matmul

In [13]:

```
fp8_max = 448.0
def quantize_block(block):
    amax = np.max(np.abs(block))
    scale = fp8_max/amax
    scaled = block*scale
    sign = np.sign(scaled)
    absx = np.abs(scaled); absx = np.where(absx==0, 1e-12, absx)
    exp = np.floor(np.log2(absx))
    mant = absx/(2**exp)
    mant_q = np.round(mant*8)/8
    quant = sign*mant_q*(2**exp)
    return quant/scale, scale

Wo_quant, scale = quantize_block(Wo)
print(f"Block scale = {fp8_max}/{np.max(np.abs(Wo)):.4f} = {scale:.4f}")
print("Wo original:\n", Wo)
print("Wo after FP8 quantize->dequantize:\n", Wo_quant)

out_full = concat @ Wo
out_quant = concat @ Wo_quant
print("\nMultiHead(X) full precision:\n", out_full)
print("MultiHead(X) FP8-quantized:\n", out_quant)
print("Difference (the real 'cost' of FP8):\n", np.abs(out_full-out_quant))
```

Block scale = 448.0/0.5000 = 896.0000

Wo original:

```
[[0.5 0.2 0.1 0.3]
 [0.3 0.4 0.2 0.1]]
```

Wo after FP8 quantize->dequantize:

```
[[0.5    0.1964 0.0982 0.2857]
 [0.2857 0.3929 0.1964 0.0982]]
```

MultiHead(X) full precision:

```
[[0.238 0.168 0.084 0.122 ]
 [0.1619 0.0964 0.0482 0.0881]
 [0.1069 0.0589 0.0294 0.0595]
 [0.1475 0.1013 0.0506 0.0764]]
```

MultiHead(X) FP8-quantized:

```
[[0.2342 0.165 0.0825 0.1169]
 [0.1603 0.0947 0.0473 0.0842]
 [0.1061 0.0578 0.0289 0.0569]
 [0.1454 0.0995 0.0497 0.0732]]
```

Difference (the real 'cost' of FP8):

```
[[0.0037 0.003 0.0015 0.005 ]
 [0.0016 0.0017 0.0009 0.0039]
 [0.0008 0.0011 0.0005 0.0027]
 [0.0022 0.0018 0.0009 0.0032]]
```

Exercise: try it yourself

1. **Change num_heads or nope_dim/rope_dim** and confirm the shapes still work through MLA.
2. **Add a 5th routed expert** and see how routing/gating changes for each token.
3. **Verify near-uniform attention** for an untrained model: replace the Wq/Wk-equivalent projections with small random values (e.g. `np.random.uniform(-0.05,0.05,...)`) and confirm the causal softmax weights become close to uniform over each token's visible positions.
4. **Stack a second decoder layer**: feed `final_output` back in as the new `X` and re-run from Step 3 onward (skip the embedding lookup, it only happens once).
5. **Change the FP8 block size**: quantize `Wo` split into two separate 1x4 blocks (one per row) instead of one 2x4 block, and compare the resulting error — this is the actual tile-wise vs. block-wise trade-off DeepSeek-V3 makes for activations vs. weights.

In [14]:

```
# Your experiments here
```